Now when you issue a `GET` request to the /health endpoint, you get full health indicator details. Here's a sample of what that might look like for a service that integrates with the Mongo document database:

```
{
  "status": "UP",
  "details": {
    "mongo": {
      "status": "UP",
      "details": {
        "version": "3.5.5"
      }
    },
    "diskSpace": {
      "status": "UP",
      "details": {
        "total": 499963170816,
        "free": 177284784128,
        "threshold": 10485760
      }
    }
  }
}
```

All applications, regardless of any other external dependencies, will have a health indicator for the filesystem named `diskSpace`. The `diskSpace` health indicator indicates the health of the filesystem (hopefully, `UP`), which is determined by how much free space is remaining. If the available disk space drops below the threshold, it will report a status of `DOWN`.

In the preceding example, there's also a `mongo` health indicator, which reports the status of the Mongo database. Details shown include the Mongo database version.

Autoconfiguration ensures that only health indicators that are pertinent to an application will appear in the response from the /health endpoint. In addition to the `mongo` and `diskSpace` health indicators, Spring Boot also provides health indicators for several other external databases and systems, including the following:

- Cassandra
- Config Server
- Couchbase
- Eureka
- Hystrix
- JDBC data sources
- Elasticsearch
- InfluxDB
- JMS message brokers
- LDAP
- Email servers

- Neo4j
- Rabbit message brokers
- Redis
- Solr

Additionally, third-party libraries may contribute their own health indicators. We'll look at how to write a custom health indicator in section 15.3.2.

As you've seen, the /health and /info endpoints provide general information about the running application. Meanwhile, other Actuator endpoints provide insight into the application configuration. Let's look at how Actuator can show how an application is configured.

15.2.2 Viewing configuration details

Beyond receiving general information about an application, it can be enlightening to understand how an application is configured. What beans are in the application context? What autoconfiguration conditions passed or failed? What environment properties are available to the application? How are HTTP requests mapped to controllers? What logging level are one or more packages or classes set to?

These questions are answered by Actuator's /beans, /conditions, /env, /configprops, /mappings, and /loggers endpoints. And in some cases, such as /env and /loggers, you can even adjust the configuration of a running application on the fly. We'll look at how each of these endpoints gives insight into the configuration of a running application, starting with the /beans endpoint.

GETTING A BEAN WIRING REPORT

The most essential endpoint for exploring the Spring application context is the /beans endpoint. This endpoint returns a JSON document describing every single bean in the application context, its Java type, and any of the other beans it's injected with.

A complete response from a `GET` request to /beans could easily fill this entire chapter. Instead of examining the complete response from /beans, let's consider the following snippet, which focuses on a single bean entry:

```
{
  "contexts": {
    "application-1": {
      "beans": {
...
        "ingredientsController": {
          "aliases": [],
          "scope": "singleton",
          "type": "tacos.ingredients.IngredientsController",
          "resource": "file [/Users/habuma/Documents/Workspaces/
            TacoCloud/ingredient-service/target/classes/tacos/
            ingredients/IngredientsController.class]",
          "dependencies": [
            "ingredientRepository"
```

```
          ]
        },
...
      },
      "parentId": null
    }
  }
}
```

At the root of the response is the `contexts` element, which includes one subelement for each Spring application context in the application. Within each application context is a `beans` element that holds details for all the beans in the application context.

In the preceding example, the bean shown is the one whose name is `ingredients-Controller`. You can see that it has no aliases, is scoped as a singleton, and is of type `tacos.ingredients.IngredientsController`. Moreover, the `resource` property gives the path to the class file that defines the bean. And the `dependencies` property lists all other beans that are injected into the given bean. In this case, the `ingredients-Controller` bean is injected with a bean whose name is `ingredientRepository`.

EXPLAINING AUTOCONFIGURATION

As you've seen, autoconfiguration is one of the most powerful things that Spring Boot offers. Sometimes, however, you may wonder why something has been autoconfigured. Or you may expect something to have been autoconfigured and are left wondering why it hasn't been. In that case, you can make a `GET` request to /conditions to get an explanation of what took place in autoconfiguration.

The autoconfiguration report returned from /conditions is divided into three parts: positive matches (conditional configuration that passed), negative matches (conditional configuration that failed), and unconditional classes. The following snippet from the response to a request to /conditions shows an example of each section:

```
{
  "contexts": {
    "application-1": {
      "positiveMatches": {
...
        "MongoDataAutoConfiguration#mongoTemplate": [
          {
            "condition": "OnBeanCondition",
            "message": "@ConditionalOnMissingBean (types:
                org.springframework.data.mongodb.core.MongoTemplate;
                SearchStrategy: all) did not find any beans"
          }
        ],
...
      },
      "negativeMatches": {
...
        "DispatcherServletAutoConfiguration": {
          "notMatched": [
```

```
            {
              "condition": "OnClassCondition",
              "message": "@ConditionalOnClass did not find required
                  class 'org.springframework.web.servlet.
                                                  DispatcherServlet'"
            }
          ],
          "matched": []
        },
...
      },
      "unconditionalClasses": [
...
        "org.springframework.boot.autoconfigure.context.
                        ConfigurationPropertiesAutoConfiguration",
...
      ]
    }
  }
}
```

Under the `positiveMatches` section, you see that a `MongoTemplate` bean was configured by autoconfiguration because one didn't already exist. The autoconfiguration that caused this includes a `@ConditionalOnMissingBean` annotation, which passes off the bean to be configured if it hasn't already been explicitly configured. But in this case, no beans of type `MongoTemplate` were found, so autoconfiguration stepped in and configured one.

Under `negativeMatches`, Spring Boot autoconfiguration considered configuring a `DispatcherServlet`. But the `@ConditionalOnClass` conditional annotation failed because `DispatcherServlet` couldn't be found.

Finally, a `ConfigurationPropertiesAutoConfiguration` bean was configured unconditionally, as seen under the `unconditionalClasses` section. Configuration properties are foundational to how Spring Boot operates, so you should autoconfigure any configuration pertaining to configuration properties without any conditions.

Inspecting the environment and configuration properties

In addition to knowing how your application beans are wired together, you might also be interested in learning what environment properties are available and what configuration properties were injected into the beans.

When you issue a `GET` request to the `/env` endpoint, you'll receive a rather lengthy response that includes properties from all property sources in play in the Spring application. This includes properties from environment variables, JVM system properties, application.properties and application.yml files, and even the Spring Cloud Config Server (if the application is a client of the Config Server).

The following listing shows a greatly abridged example of the kind of response you might get from the `/env` endpoint, to give you some idea of the kind of information it provides.

Listing 15.1 The results from the /env endpoint

```
$ curl localhost:8080/actuator/env
{
  "activeProfiles": [
    "development"
  ],
  "propertySources": [
...
    {
      "name": "systemEnvironment",
      "properties": {
        "PATH": {
          "value": "/usr/bin:/bin:/usr/sbin:/sbin",
          "origin": "System Environment Property \"PATH\""
        },
...
        "HOME": {
          "value": "/Users/habuma",
          "origin": "System Environment Property \"HOME\""
        }
      }
    },
    {
      "name": "applicationConfig: [classpath:/application.yml]",
      "properties": {
        "spring.application.name": {
          "value": "ingredient-service",
          "origin": "class path resource [application.yml]:3:11"
        },
        "server.port": {
          "value": 8080,
          "origin": "class path resource [application.yml]:9:9"
        },
...
      }
    },
...
  ]
}
```

Although the full response from /env provides even more information, what's shown in listing 15.1 contains a few noteworthy elements. First, notice that near the top of the response is a field named `activeProfiles`. In this case, it indicates that the `development` profile is active. If any other profiles were active, those would be listed as well.

Next, the `propertySources` field is an array containing an entry for every property source in the Spring application environment. In listing 15.1, only the `systemEnvironment` and an `applicationConfig` property source referencing the application.yml file are shown.

Within each property source is a listing of all properties provided by that source, paired with their values. In the case of the application.yml property source, the `origin`

field for each property tells exactly where the property is set, including the line and column within application.yml.

The /env endpoint can also be used to fetch a specific property when that property's name is given as the second element of the path. For example, to examine the `server.port` property, submit a `GET` request for /env/server.port, as shown here:

```
$ curl localhost:8080/actuator/env/server.port
{
  "property": {
    "source": "systemEnvironment", "value": "8080"
  },
  "activeProfiles": [ "development" ],
  "propertySources": [
    { "name": "server.ports" },
    { "name": "mongo.ports" },
    { "name": "systemProperties" },
    { "name": "systemEnvironment",
      "property": {
        "value": "8080",
        "origin": "System Environment Property \"SERVER_PORT\""
      }
    },
    { "name": "random" },
    { "name": "applicationConfig: [classpath:/application.yml]",
      "property": {
        "value": 0,
        "origin": "class path resource [application.yml]:9:9"
      }
    },
    { "name": "springCloudClientHostInfo" },
    { "name": "refresh" },
    { "name": "defaultProperties" },
    { "name": "Management Server" }
  ]
}
```

As you can see, all property sources are still represented, but only those that set the specified property will contain any additional information. In this case, both the `systemEnvironment` property source and the application.yml property source had values for the `server.port` property. Because the `systemEnvironment` property source takes precedence over any of the property sources listed below it, its value of `8080` wins. The winning value is reflected near the top under the `property` field.

The /env endpoint can be used for more than just reading property values. By submitting a `POST` request to the /env endpoint, along with a JSON document with `name` and `value` fields, you can also set properties in the running application. For example, to set a property named `tacocloud.discount.code` to `TACOS1234`, you can use `curl` to submit the `POST` request at the command line like this:

```
$ curl localhost:8080/actuator/env \
       -d'{"name":"tacocloud.discount.code","value":"TACOS1234"}' \
```

```
       -H "Content-type: application/json"
{"tacocloud.discount.code":"TACOS1234"}
```

After submitting the property, the newly set property and its value are returned in the response. Later, should you decide you no longer need that property, you can submit a `DELETE` request to the /env endpoint as follows to delete all properties created through that endpoint:

```
$ curl localhost:8080/actuator/env -X DELETE
{"tacocloud.discount.code":"TACOS1234"}
```

As useful as setting properties through Actuator's API can be, it's important to be aware that any properties set with a `POST` request to the /env endpoint apply only to the application instance receiving the request, are temporary, and will be lost when the application restarts.

NAVIGATING HTTP REQUEST MAPPINGS

Although Spring MVC's (and Spring WebFlux's) programming model makes it easy to handle HTTP requests by simply annotating methods with request-mapping annotations, it can sometimes be challenging to get a big-picture understanding of all the kinds of HTTP requests that an application can handle and what kinds of components handle those requests.

Actuator's /mappings endpoint offers a one-stop view of every HTTP request handler in an application, whether it be from a Spring MVC controller or one of Actuator's own endpoints. To get a complete list of all the endpoints in a Spring Boot application, make a `GET` request to the /mappings endpoint, and you might receive something that's a little bit like the abridged response shown next.

Listing 15.2 HTTP mappings as shown by the /mappings endpoint

```
$ curl localhost:8080/actuator/mappings | jq
{
  "contexts": {
    "application-1": {
      "mappings": {
        "dispatcherHandlers": {
          "webHandler": [
...
            {
              "predicate": "{[/ingredients],methods=[GET]}",
              "handler": "public
     reactor.core.publisher.Flux<tacos.ingredients.Ingredient>
     tacos.ingredients.IngredientsController.allIngredients()",
              "details": {
                "handlerMethod": {
                  "className": "tacos.ingredients.IngredientsController",
                  "name": "allIngredients",
                  "descriptor": "()Lreactor/core/publisher/Flux;"
                },
```

```
                "handlerFunction": null,
                "requestMappingConditions": {
                  "consumes": [],
                  "headers": [],
                  "methods": [
                    "GET"
                  ],
                  "params": [],
                  "patterns": [
                    "/ingredients"
                  ],
                  "produces": []
                }
              }
            },
...
          ]
        }
      },
      "parentId": "application-1"
    },
    "bootstrap": {
      "mappings": {
        "dispatcherHandlers": {}
      },
      "parentId": null
    }
  }
}
```

Here, the response from the `curl` command line is piped to a utility called `jq` (https://stedolan.github.io/jq/), which, among other things, pretty-prints the JSON returned from the request in an easily readable format. For the sake of brevity, this response has been abridged to show only a single request handler. Specifically, it shows that `GET` requests for /ingredients will be handled by the `allIngredients()` method of `IngredientsController`.

Managing logging levels

Logging is an important feature of any application. Logging can provide a means of auditing as well as a crude means of debugging.

Setting logging levels can be quite a balancing act. If you set the logging level to be too verbose, there may be too much noise in the logs, and finding useful information may be difficult. On the other hand, if you set logging levels to be too slack, the logs may not be of much value in understanding what an application is doing.

Logging levels are typically applied on a package-by-package basis. If you're ever wondering what logging levels are set in your running Spring Boot application, you can issue a `GET` request to the /loggers endpoint. The following JSON code shows an excerpt from a response to /loggers:

```
{
  "levels": [ "OFF", "ERROR", "WARN", "INFO", "DEBUG", "TRACE" ],
  "loggers": {
    "ROOT": {
      "configuredLevel": "INFO", "effectiveLevel": "INFO"
    },
...
    "org.springframework.web": {
      "configuredLevel": null, "effectiveLevel": "INFO"
    },
...
    "tacos": {
      "configuredLevel": null, "effectiveLevel": "INFO"
    },
    "tacos.ingredients": {
      "configuredLevel": null, "effectiveLevel": "INFO"
    },
    "tacos.ingredients.IngredientServiceApplication": {
      "configuredLevel": null, "effectiveLevel": "INFO"
    }
  }
}
```

The response starts off with a list of all valid logging levels. After that, the `loggers` element lists logging-level details for each package in the application. The `configuredLevel` property shows the logging level that has been explicitly configured (or `null` if it hasn't been explicitly configured). The `effectiveLevel` property gives the effective logging level, which may have been inherited from a parent package or from the root logger.

Although this excerpt shows logging levels only for the root logger and four packages, the complete response will include logging-level entries for every single package in the application, including those for libraries that are in use. If you'd rather focus your request on a specific package, you can specify the package name as an extra path component in the request.

For example, if you just want to know what logging levels are set for the `tacocloud.ingredients` package, you can make a request to /loggers/tacos.ingredients as follows:

```
{
  "configuredLevel": null,
  "effectiveLevel": "INFO"
}
```

Aside from returning the logging levels for the application packages, the /loggers endpoint also allows you to change the configured logging level by issuing a `POST` request. For example, suppose you want to set the logging level of the `tacocloud.ingredients` package to `DEBUG`. The following `curl` command will achieve that:

```
$ curl localhost:8080/actuator/loggers/tacos/ingredients \
       -d'{"configuredLevel":"DEBUG"}' \
       -H"Content-type: application/json"
```

Now that the logging level has been changed, you can issue a GET request to /loggers/tacos/ingredients as shown here to see that it has been changed:

```
{
  "configuredLevel": "DEBUG",
  "effectiveLevel": "DEBUG"
}
```

Notice that where the configuredLevel was previously null, it's now DEBUG. That change carries over to the effectiveLevel as well. But what's most important is that if any code in that package logs anything at debug level, the log files will include that debug-level information.

15.2.3 Viewing application activity

It can be useful to keep an eye on activity in a running application, including the kinds of HTTP requests that the application is handling and the activity of all of the threads in the application. For this, Actuator provides the /httptrace, /threaddump, and /heapdump endpoints.

The /heapdump endpoint is perhaps the most difficult Actuator endpoint to describe in any detail. Put succinctly, it downloads a gzip-compressed HPROF heap dump file that can be used to track down memory or thread issues. For the sake of space and because use of the heap dump is a rather advanced feature, I'm going to limit coverage of the /heapdump endpoint to this paragraph.

Tracing HTTP activity

The /httptrace endpoint reports details on the most recent 100 requests handled by an application. Details included are the request method and path, a timestamp indicating when the request was handled, headers from both the request and the response, and the time taken handling the request.

The following snippet of JSON code shows a single entry from the response of the /httptrace endpoint:

```
{
  "traces": [
    {
      "timestamp": "2020-06-03T23:41:24.494Z",
      "principal": null,
      "session": null,
      "request": {
        "method": "GET",
        "uri": "http://localhost:8080/ingredients",
        "headers": {
          "Host": ["localhost:8080"],
          "User-Agent": ["curl/7.54.0"],
          "Accept": ["*/*"]
        },
        "remoteAddress": null
      },
```

```
      "response": {
        "status": 200,
        "headers": {
          "Content-Type": ["application/json;charset=UTF-8"]
        }
      },
      "timeTaken": 4
    },
...
  ]
}
```

Although this information may be useful for debugging purposes, it's even more interesting when the trace data is tracked over time, providing insight into how busy the application was at any given time as well as how many requests were successful compared to how many failed, based on the value of the response status. In chapter 16, you'll see how Spring Boot Admin captures this information into a running graph that visualizes the HTTP trace information over a period of time.

MONITORING THREADS

In addition to HTTP request tracing, thread activity can also be useful in determining what's going on in a running application. The /threaddump endpoint produces a snapshot of current thread activity. The following snippet from a /threaddump response gives a taste of what this endpoint provides:

```
{
  "threadName": "reactor-http-nio-8",
  "threadId": 338,
  "blockedTime": -1,
  "blockedCount": 0,
  "waitedTime": -1,
  "waitedCount": 0,
  "lockName": null,
  "lockOwnerId": -1,
  "lockOwnerName": null,
  "inNative": true,
  "suspended": false,
  "threadState": "RUNNABLE",
  "stackTrace": [
    {
      "methodName": "kevent0",
      "fileName": "KQueueArrayWrapper.java",
      "lineNumber": -2,
      "className": "sun.nio.ch.KQueueArrayWrapper",
      "nativeMethod": true
    },
    {
      "methodName": "poll",
      "fileName": "KQueueArrayWrapper.java",
      "lineNumber": 198,
      "className": "sun.nio.ch.KQueueArrayWrapper",
```

```
      "nativeMethod": false
    },
...
  ],
  "lockedMonitors": [
    {
      "className": "io.netty.channel.nio.SelectedSelectionKeySet",
      "identityHashCode": 1039768944,
      "lockedStackDepth": 3,
      "lockedStackFrame": {
        "methodName": "lockAndDoSelect",
        "fileName": "SelectorImpl.java",
        "lineNumber": 86,
        "className": "sun.nio.ch.SelectorImpl",
        "nativeMethod": false
      }
    },
...
  ],
  "lockedSynchronizers": [],
  "lockInfo": null
}
```

The complete thread dump report includes every thread in the running application. To save space, the thread dump here shows an abridged entry for a single thread. As you can see, it includes details regarding the blocking and locking status of the thread, among other thread specifics. There's also a stack trace that gives some insight into which area of the code the thread is spending time on.

Because the /threaddump endpoint provides a snapshot of thread activity only at the time it was requested, it can be difficult to get a full picture of how threads are behaving over time. In chapter 16, you'll see how Spring Boot Admin can monitor the /threaddump endpoint in a live view.

15.2.4 Tapping runtime metrics

The /metrics endpoint can report many metrics produced by a running application, including memory, processor, garbage collection, and HTTP requests. Actuator provides more than two dozen categories of metrics out of the box, as evidenced by the following list of metrics categories returned when issuing a `GET` request to /metrics:

```
$ curl localhost:8080/actuator/metrics | jq
{
  "names": [
    "jvm.memory.max",
    "process.files.max",
    "jvm.gc.memory.promoted",
    "http.server.requests",
    "system.load.average.1m",
    "jvm.memory.used",
    "jvm.gc.max.data.size",
    "jvm.memory.committed",
```

```
    "system.cpu.count",
    "logback.events",
    "jvm.buffer.memory.used",
    "jvm.threads.daemon",
    "system.cpu.usage",
    "jvm.gc.memory.allocated",
    "jvm.threads.live",
    "jvm.threads.peak",
    "process.uptime",
    "process.cpu.usage",
    "jvm.classes.loaded",
    "jvm.gc.pause",
    "jvm.classes.unloaded",
    "jvm.gc.live.data.size",
    "process.files.open",
    "jvm.buffer.count",
    "jvm.buffer.total.capacity",
    "process.start.time"
  ]
}
```

So many metrics are covered that it would be impossible to discuss them all in any meaningful way in this chapter. Instead, let's focus on one category of metrics, `http.server.requests`, as an example of how to consume the /metrics endpoint.

If instead of simply requesting /metrics, you were to issue a `GET` request for /metrics/{metrics name}, you'd receive more detail about the metrics for that category. In the case of `http.server.requests`, a `GET` request for /metrics/ http.server .requests returns data that looks like the following:

```
$ curl localhost:8080/actuator/metrics/http.server.requests
{
  "name": "http.server.requests",
  "measurements": [
    { "statistic": "COUNT", "value": 2103 },
    { "statistic": "TOTAL_TIME", "value": 18.086334315 },
    { "statistic": "MAX", "value": 0.028926313 }
  ],
  "availableTags": [
    { "tag": "exception",
      "values": [ "ResponseStatusException",
                  "IllegalArgumentException", "none" ] },
    { "tag": "method", "values": [ "GET" ] },
    { "tag": "uri",
      "values": [
        "/actuator/metrics/{requiredMetricName}",
        "/actuator/health", "/actuator/info", "/ingredients",
        "/actuator/metrics", "/**" ] },
    { "tag": "status", "values": [ "404", "500", "200" ] }
  ]
}
```

The most significant portion of this response is the `measurements` section, which includes all the metrics for the requested category. In this case, it reports that there

have been 2,103 HTTP requests. The total time spent handling those requests is 18.086334315 seconds, and the maximum time spent processing any request is 0.028926313 seconds.

Those generic metrics are interesting, but you can narrow down the results further by using the tags listed under `availableTags`. For example, you know that there have been 2,103 requests, but what's unknown is how many of them resulted in an HTTP 200 versus an HTTP 404 or HTTP 500 response `status`. Using the status tag, you can get metrics for all requests resulting in an HTTP 404 status like this:

```
$ curl localhost:8080/actuator/metrics/http.server.requests? \
                                        tag=status:404
{
  "name": "http.server.requests",
  "measurements": [
    { "statistic": "COUNT", "value": 31 },
    { "statistic": "TOTAL_TIME", "value": 0.522061212 },
    { "statistic": "MAX", "value": 0 }
  ],
  "availableTags": [
    { "tag": "exception",
      "values": [ "ResponseStatusException", "none" ] },
    { "tag": "method", "values": [ "GET" ] },
    { "tag": "uri",
      "values": [
        "/actuator/metrics/{requiredMetricName}", "/**" ] }
  ]
}
```

By specifying the tag name and value with the `tag` request attribute, you now see metrics specifically for requests that resulted in an HTTP 404 response. This shows that there were 31 requests resulting in a 404, and it took 0.522061212 seconds to serve them all. Moreover, it's clear that some of the failing requests were `GET` requests for /actuator/metrics/{requiredMetricsName} (although it's unclear what the {requiredMetricsName} path variable resolved to). And some were for some other path, captured by the `/**` wildcard path.

Hmmm . . . what if you want to know how many of those HTTP 404 responses were for the /** path? All you need to do to filter this further is to specify the `uri` tag in the request, like this:

```
% curl "localhost:8080/actuator/metrics/http.server.requests? \
                                     tag=status:404&tag=uri:/**"
{
  "name": "http.server.requests",
  "measurements": [
    { "statistic": "COUNT", "value": 30 },
    { "statistic": "TOTAL_TIME", "value": 0.519791548 },
    { "statistic": "MAX", "value": 0 }
  ],
  "availableTags": [
    { "tag": "exception", "values": [ "ResponseStatusException" ] },
```

```
    { "tag": "method", "values": [ "GET" ] }
  ]
}
```

Now you can see that there were 30 requests for some path that matched /** that resulted in an HTTP 404 response, and it took a total of 0.519791548 seconds to handle those requests.

You'll also notice that as you refine the request, the available tags are more limited. The tags offered are only those that match the requests captured by the displayed metrics. In this case, the `exception` and `method` tags each have only a single value; it's obvious that all 30 of the requests were `GET` requests that resulted in a 404 because of a `ResponseStatusException`.

Navigating the /metrics endpoint can be a tricky business, but with a little practice, it's not impossible to get the data you're looking for. In chapter 16, you'll see how Spring Boot Admin makes consuming data from the /metrics endpoint much easier.

Although the information presented by Actuator endpoints offers useful insight into the inner workings of a running Spring Boot application, it's not well suited for human consumption. Because Actuator endpoints are REST endpoints, the data they provide is intended for consumption by some other application, perhaps a UI. With that in mind, let's see how you can present Actuator information in a user-friendly web application.

15.3 Customizing Actuator

One of the greatest features of Actuator is that it can be customized to meet the specific needs of an application. A few of the endpoints themselves allow for customization. Meanwhile, Actuator itself allows you to create custom endpoints.

Let's look at a few ways that Actuator can be customized, starting with ways to add information to the /info endpoint.

15.3.1 Contributing information to the /info endpoint

As you saw in section 15.2.1, the /info endpoint starts off empty and uninformative. But you can easily add data to it by creating properties that are prefixed with `info.`

Although prefixing properties with `info.` is a very easy way to get custom data into the /info endpoint, it's not the only way. Spring Boot offers an interface named `InfoContributor` that allows you to programmatically add any information you want to the /info endpoint response. Spring Boot even comes ready with a couple of useful implementations of `InfoContributor` that you'll no doubt find useful.

Let's see how you can write your own `InfoContributor` to add some custom info to the /info endpoint.

CREATING A CUSTOM INFOCONTRIBUTOR

Suppose you want to add some simple statistics regarding Taco Cloud to the /info endpoint. For example, let's say you want to include information about how many

tacos have been created. To do that, you can write a class that implements `InfoContributor`, inject it with `TacoRepository`, and then publish whatever count that `TacoRepository` gives you as information to the /info endpoint. The next listing shows how you might implement such a contributor.

Listing 15.3 A custom implementation of `InfoContributor`

```
package tacos.actuator;

import java.util.HashMap;
import java.util.Map;

import org.springframework.boot.actuate.info.Info.Builder;
import org.springframework.boot.actuate.info.InfoContributor;
import org.springframework.stereotype.Component;

import tacos.data.TacoRepository;

@Component
public class TacoCountInfoContributor implements InfoContributor {
  private TacoRepository tacoRepo;

  public TacoCountInfoContributor(TacoRepository tacoRepo) {
    this.tacoRepo = tacoRepo;
  }

  @Override
  public void contribute(Builder builder) {
    long tacoCount = tacoRepo.count().block();
    Map<String, Object> tacoMap = new HashMap<String, Object>();
    tacoMap.put("count", tacoCount);
    builder.withDetail("taco-stats", tacoMap);
  }
}
```

By implementing `InfoContributor`, `TacoCountInfoContributor` is required to implement the `contribute()` method. This method is given a `Builder` object on which the `contribute()` method makes a call to `withDetail()` to add info details. In your implementation, you consult `TacoRepository` by calling its `count()` method to find out how many tacos have been created. In this particular case, you're working with a reactive repository, so you need to call `block()` to get the count out of the returned `Mono<Long>`. Then you put that count into a `Map`, which you then give to the builder with the label `taco-stats`. The results of the /info endpoint will include that count, as shown here:

```
{
  "taco-stats": {
    "count": 44
  }
}
```

As you can see, an implementation of `InfoContributor` is able to use whatever means necessary to contribute information. This is in contrast to simply prefixing a property with `info.`, which, although simple, is limited to static values.

INJECTING BUILD INFORMATION INTO THE /INFO ENDPOINT

Spring Boot comes with a few built-in implementations of `InfoContributor` that automatically add information to the results of the /info endpoint. Among them is `BuildInfoContributor`, which adds information from the project build file into the /info endpoint results. The basic information includes the project version, the timestamp of the build, and the host and user who performed the build.

To enable build information to be included in the results of the /info endpoint, add the `build-info` goal to the Spring Boot Maven Plugin executions, as follows:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <executions>
        <execution>
          <goals>
            <goal>build-info</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

If you're using Gradle to build your project, you can simply add the following lines to your build.gradle file:

```
springBoot {
  buildInfo()
}
```

In either event, the build will produce a file named build-info.properties in the distributable JAR or WAR file that `BuildInfoContributor` will consume and contribute to the /info endpoint. The following snippet from the /info endpoint response shows the build information that's contributed:

```
{
  "build": {
    "artifact": "tacocloud",
    "name": "taco-cloud",
    "time": "2021-08-08T23:55:16.379Z",
    "version": "0.0.15-SNAPSHOT",
    "group": "sia"
  },
}
```

This information is useful for understanding exactly which version of an application is running and when it was built. By performing a `GET` request to the /info endpoint, you'll know whether you're running the latest and greatest build of the project.

Exposing Git commit information

Assuming that your project is kept in Git for source code control, you may want to include Git commit information in the /info endpoint. To do that, you'll need to add the following plugin in the Maven project pom.xml:

```
<build>
  <plugins>
...
    <plugin>
      <groupId>pl.project13.maven</groupId>
      <artifactId>git-commit-id-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

If you're a Gradle user, don't worry. There's an equivalent plugin for you to add to your build.gradle file, shown here:

```
plugins {
  id "com.gorylenko.gradle-git-properties" version "2.3.1"
}
```

Both of these plugins do essentially the same thing: they generate a build-time artifact named git.properties that contains all of the Git metadata for the project. A special `InfoContributor` implementation discovers that file at runtime and exposes its contents as part of the /info endpoint.

Of course, to generate the git.properties file, the project needs to have Git commit metadata. That is, it must be a clone of a Git repository or be a newly initialized local Git repository with at least one commit. If not, then either of these plugins will fail. You can, however, configure them to ignore the missing Git metadata. For the Maven plugin, set the `failOnNoGitDirectory` property to `false` like this:

```
<build>
  <plugins>
...
    <plugin>
      <groupId>pl.project13.maven</groupId>
      <artifactId>git-commit-id-plugin</artifactId>
      <configuration>
        <failOnNoGitDirectory>false</failOnNoGitDirectory>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Similarly, you can set the `failOnNoGitDirectory` property in Gradle by specifying it under `gitProperties` like this:

```
gitProperties {
  failOnNoGitDirectory = false
}
```

In its simplest form, the Git information presented in the /info endpoint includes the Git branch, commit hash, and timestamp that the application was built against, as shown here:

```
{
  "git": {
    "branch": "main",
    "commit": {
      "id": "df45505",
      "time": "2021-08-08T21:51:12Z"
    }
  },
...
}
```

This information is quite definitive in describing the state of the code when the project was built. But by setting the `management.info.git.mode` property to `full`, you can get extremely detailed information about the Git commit that was in play when the project was built, as shown in the next code sample:

```
management:
  info:
    git:
      mode: full
```

The following listing shows a sample of what the full Git info might look like.

Listing 15.4 Full Git commit info exposed through the /info endpoint

```
"git": {
  "local": {
    "branch": {
      "ahead": "8",
      "behind": "0"
    }
  },
  "commit": {
    "id": {
      "describe-short": "df45505-dirty",
      "abbrev": "df45505",
      "full": "df455055daaf3b1347b0ad1d9dca4ebbc6067810",
      "describe": "df45505-dirty"
    },
    "message": {
      "short": "Apply chapter 18 edits",
```

```
      "full": "Apply chapter 18 edits"
    },
    "user": {
      "name": "Craig Walls",
      "email": "craig@habuma.com"
    },
    "author": {
      "time": "2021-08-08T15:51:12-0600"
    },
    "committer": {
      "time": "2021-08-08T15:51:12-0600"
    },
    "time": "2021-08-08T21:51:12Z"
  },
  "branch": "master",
  "build": {
    "time": "2021-08-09T00:13:37Z",
    "version": "0.0.15-SNAPSHOT",
    "host": "Craigs-MacBook-Pro.local",
    "user": {
      "name": "Craig Walls",
      "email": "craig@habuma.com"
    }
  },
  "tags": "",
  "total": {
    "commit": {
      "count": "196"
    }
  },
  "closest": {
    "tag": {
      "commit": {
        "count": ""
      },
      "name": ""
    }
  },
  "remote": {
    "origin": {
      "url": "git@github.com:habuma/spring-in-action-6-samples.git"
    }
  },
  "dirty": "true"
},
```


In addition to the timestamp and abbreviated Git commit hash, the full version includes the name and email of the user who committed the code as well as the commit message and other information, allowing you to pinpoint exactly what code was used to build the project. In fact, notice that the `dirty` field in listing 15.4 is `true`, indicating that some uncommitted changes existed in the build directory when the project was built. It doesn't get much more definitive than that!

15.3.2 Defining custom health indicators

Spring Boot comes with several out-of-the-box health indicators that provide health information for many common external systems that a Spring application may integrate with. But at some point, you may find that you're interacting with some external system that Spring Boot neither anticipated nor provided a health indicator for.

For instance, your application may integrate with a legacy mainframe application, and the health of your application may be affected by the health of the legacy system. To create a custom health indicator, all you need to do is create a bean that implements the `HealthIndicator` interface.

As it turns out, the Taco Cloud services have no need for a custom health indicator, because the ones provided by Spring Boot are more than sufficient. But to demonstrate how you can develop a custom health indicator, consider the next listing, which shows a simple implementation of `HealthIndicator` in which health is determined somewhat randomly by the time of day.

Listing 15.5 An unusual implementation of `HealthIndicator`

```
package tacos.actuator;

import java.util.Calendar;
import org.springframework.boot.actuate.health.Health;
import org.springframework.boot.actuate.health.HealthIndicator;
import org.springframework.stereotype.Component;


@Component
public class WackoHealthIndicator
       implements HealthIndicator {
  @Override
  public Health health() {
    int hour = Calendar.getInstance().get(Calendar.HOUR_OF_DAY);
    if (hour > 12) {
      return Health
          .outOfService()
          .withDetail("reason",
                 "I'm out of service after lunchtime")
          .withDetail("hour", hour)
          .build();
    }

    if (Math.random() <= 0.1) {
      return Health
          .down()
          .withDetail("reason", "I break 10% of the time")
          .build();
    }
    return Health
        .up()
        .withDetail("reason", "All is good!")
```

```
            .build();
    }
}
```

This crazy health indicator first checks what the current time is, and if it's after noon, returns a health status of `OUT_OF_SERVICE`, with a few details explaining the reason for that status. Even if it's before lunch, there's a 10% chance that the health indicator will report a `DOWN` status, because it uses a random number to decide whether or not it's up. If the random number is less than 0.1, the status will be reported as `DOWN`. Otherwise, the status will be `UP`.

Obviously, the health indicator in listing 15.5 isn't going to be very useful in any real-world applications. But imagine that instead of consulting the current time or a random number, it were to make a remote call to some external system and determine the status based on the response it receives. In that case, it would be a very useful health indicator.

15.3.3 Registering custom metrics

In section 15.2.4, we looked at how you could navigate the /metrics endpoint to consume various metrics published by Actuator, with a focus on metrics pertaining to HTTP requests. The metrics provided by Actuator are very useful, but the /metrics endpoint isn't limited to only those built-in metrics.

Ultimately, Actuator metrics are implemented by Micrometer (https://micrometer.io/), a vendor-neutral metrics facade that makes it possible for applications to publish any metrics they want and to display them in the third-party monitoring system of their choice, including support for Prometheus, Datadog, and New Relic, among others.

The most basic means of publishing metrics with Micrometer is through Micrometer's `MeterRegistry`. In a Spring Boot application, all you need to do to publish metrics is inject a `MeterRegistry` wherever you may need to publish counters, timers, or gauges that capture the metrics for your application.

As an example of publishing custom metrics, suppose you want to keep counters for the numbers of tacos that have been created with different ingredients. That is, you want to track how many tacos have been made with lettuce, ground beef, flour tortillas, or any of the available ingredients. The `TacoMetrics` bean in the next listing shows how you might use `MeterRegistry` to gather that information.

Listing 15.6 `TacoMetrics` registers metrics around taco ingredients

```
package tacos.actuator;

import java.util.List;
import org.springframework.data.rest.core.event.AbstractRepositoryEventListener;
import org.springframework.stereotype.Component;
import io.micrometer.core.instrument.MeterRegistry;
import tacos.Ingredient;
import tacos.Taco;
```

```
@Component
public class TacoMetrics extends AbstractRepositoryEventListener<Taco> {
  private MeterRegistry meterRegistry;

  public TacoMetrics(MeterRegistry meterRegistry) {
    this.meterRegistry = meterRegistry;
  }

  @Override
  protected void onAfterCreate(Taco taco) {
    List<Ingredient> ingredients = taco.getIngredients();
    for (Ingredient ingredient : ingredients) {
      meterRegistry.counter("tacocloud",
          "ingredient", ingredient.getId()).increment();
    }
  }
}
```

As you can see, `TacoMetrics` is injected through its constructor with a `MeterRegistry`. It also extends `AbstractRepositoryEventListener`, a Spring Data class that enables the interception of repository events and overrides the `onAfterCreate()` method so that it can be notified any time a new `Taco` object is saved.

Within `onAfterCreate()`, a counter is declared for each `ingredient` where the tag name is ingredient and the tag value is equal to the ingredient ID. If a counter with that tag already exists, it will be reused. The counter is incremented, indicating that another taco has been created for the ingredient.

After a few tacos have been created, you can start querying the /metrics endpoint for ingredient counts. A `GET` request to /metrics/tacocloud yields some unfiltered metric counts, as shown next:

```
$ curl localhost:8080/actuator/metrics/tacocloud
{
  "name": "tacocloud",
  "measurements": [ { "statistic": "COUNT", "value": 84 }
  ],
  "availableTags": [
    {
      "tag": "ingredient",
      "values": [ "FLTO", "CHED", "LETC", "GRBF",
                  "COTO", "JACK", "TMTO", "SLSA"]
    }
  ]
}
```

The count value under `measurements` doesn't mean much here, because it's a sum of all the counts for all ingredients. But let's suppose you want to know how many tacos have been created with flour tortillas (`FLTO`). All you need to do is specify the `ingredient` tag with a value of `FLTO` as follows:

```
$ curl localhost:8080/actuator/metrics/tacocloud?tag=ingredient:FLTO

{
  "name": "tacocloud",
  "measurements": [
    { "statistic": "COUNT", "value": 39 }
  ],
  "availableTags": []
}
```

Now it's clear that 39 tacos have had flour tortillas as one of their ingredients.

15.3.4 Creating custom endpoints

At first glance, you might think that Actuator's endpoints are implemented as nothing more than Spring MVC controllers. But as you'll see in chapter 17, the endpoints are also exposed as JMX MBeans as well as through HTTP requests. Therefore, there must be something more to these endpoints than just a controller class.

In fact, Actuator endpoints are defined quite differently from controllers. Instead of a class that's annotated with `@Controller` or `@RestController`, Actuator endpoints are defined with classes that are annotated with `@Endpoint`.

What's more, instead of using HTTP-named annotations such as `@GetMapping`, `@PostMapping`, or `@DeleteMapping`, Actuator endpoint operations are defined by methods annotated with `@ReadOperation`, `@WriteOperation`, and `@DeleteOperation`. These annotations don't imply any specific communication mechanism and, in fact, allow Actuator to communicate by any variety of communication mechanisms, HTTP, and JMX out of the box. To demonstrate how to write a custom Actuator endpoint, consider `NotesEndpoint` in the next listing.

Listing 15.7 A custom endpoint for taking notes

```
package tacos.actuator;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import org.springframework.boot.actuate.endpoint.annotation.DeleteOperation;
import org.springframework.boot.actuate.endpoint.annotation.Endpoint;
import org.springframework.boot.actuate.endpoint.annotation.ReadOperation;
import org.springframework.boot.actuate.endpoint.annotation.WriteOperation;
import org.springframework.stereotype.Component;

@Component
@Endpoint(id="notes", enableByDefault=true)
public class NotesEndpoint {

  private List<Note> notes = new ArrayList<>();

  @ReadOperation
  public List<Note> notes() {
```

```
    return notes;
  }

  @WriteOperation
  public List<Note> addNote(String text) {
    notes.add(new Note(text));
    return notes;
  }

  @DeleteOperation
  public List<Note> deleteNote(int index) {
    if (index < notes.size()) {
      notes.remove(index);
    }
    return notes;
  }

  class Note {
    private Date time = new Date();
    private final String text;

    public Note(String text) {
      this.text = text;
    }

    public Date getTime() {
        return time;
    }

    public String getText() {
        return text;
    }
  }
}
```

This endpoint is a simple note-taking endpoint, wherein one can submit a note with a write operation, read the list of notes with a read operation, and remove a note with the delete operation. Admittedly, this endpoint isn't very useful as far as Actuator endpoints go. But when you consider that the out-of-the-box Actuator endpoints cover so much ground, it's difficult to come up with a practical example of a custom Actuator endpoint.

At any rate, the `NotesEndpoint` class is annotated with `@Component` so that it will be picked up by Spring's component scanning and instantiated as a bean in the Spring application context. But more relevant to this discussion, it's also annotated with `@Endpoint`, making it an Actuator endpoint with an ID of `notes`. And it's enabled by default so that you won't need to explicitly enable it by including it in the `management.web.endpoints.web.exposure.include` configuration property.

As you can see, `NotesEndpoint` offers one of each kind of operation:

- The `notes()` method is annotated with `@ReadOperation`. When invoked, it will return a list of available notes. In HTTP terms, this means it will handle an HTTP `GET` request for /actuator/notes and respond with a JSON list of notes.

- The `addNote()` method is annotated with `@WriteOperation`. When invoked, it will create a new note from the given text and add it to the list. In HTTP terms, it handles a `POST` request where the body of the request is a JSON object with a `text` property. It finishes by responding with the current state of the notes list.
- The `deleteNote()` method is annotated with `@DeleteOperation`. When invoked, it will delete the note at the given index. In HTTP terms, this endpoint handles `DELETE` requests where the index is given as a request parameter.

To see this in action, you can use `curl` to poke about with this new endpoint. First, add a couple of notes, using two separate `POST` requests, as shown here:

```
$ curl localhost:8080/actuator/notes \
               -d'{"text":"Bring home milk"}' \
               -H"Content-type: application/json"
[{"time":"2020-06-08T13:50:45.085+0000","text":"Bring home milk"}]

$ curl localhost:8080/actuator/notes \
               -d'{"text":"Take dry cleaning"}' \
               -H"Content-type: application/json"
[{"time":"2021-07-03T12:39:13.058+0000","text":"Bring home milk"},
 {"time":"2021-07-03T12:39:16.012+0000","text":"Take dry cleaning"}]
```

As you can see, each time a new note is posted, the endpoint responds with the newly appended list of notes. But if later you want to view the list of notes, you can do a simple `GET` request like so:

```
$ curl localhost:8080/actuator/notes
[{"time":"2021-07-03T12:39:13.058+0000","text":"Bring home milk"},
 {"time":"2021-07-03T12:39:16.012+0000","text":"Take dry cleaning"}]
```

If you decide to remove one of the notes, a `DELETE` request with an `index` request parameter, shown next, should do the trick:

```
$ curl localhost:8080/actuator/notes?index=1 -X DELETE
[{"time":"2021-07-03T12:39:13.058+0000","text":"Bring home milk"}]
```

It's important to note that although I've shown only how to interact with the endpoint using HTTP, it will also be exposed as an MBean that can be accessed using whatever JMX client you choose. But if you want to limit it to only exposing an HTTP endpoint, you can annotate the endpoint class with `@WebEndpoint` instead of `@Endpoint` as follows:

```
@Component
@WebEndpoint(id="notes", enableByDefault=true)
public class NotesEndpoint {
  ...
}
```

Likewise, if you prefer an MBean-only endpoint, annotate the class with `@JmxEndpoint`.

15.4 Securing Actuator

The information presented by Actuator is probably not something that you would want prying eyes to see. Moreover, because Actuator provides a few operations that let you change environment properties and logging levels, it's probably a good idea to secure Actuator so that only clients with proper access will be allowed to consume its endpoints.

Even though it's important to secure Actuator, security is outside of Actuator's responsibilities. Instead, you'll need to use Spring Security to secure Actuator. And because Actuator endpoints are just paths in the application like any other path in the application, there's nothing unique about securing Actuator versus any other application path. Everything we discussed in chapter 5 applies when securing Actuator endpoints.

Because all Actuator endpoints are gathered under a common base path of /actuator (or possibly some other base path if the `management.endpoints.web.base-path` property is set), it's easy to apply authorization rules to all Actuator endpoints across the board. For example, to require that a user have `ROLE_ADMIN` authority to invoke Actuator endpoints, you might override the `configure()` method of `WebSecurityConfigurerAdapter` like this:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
  http
    .authorizeRequests()
      .antMatchers("/actuator/**").hasRole("ADMIN")

    .and()

    .httpBasic();
}
```

This requires that all requests be from an authenticated user with `ROLE_ADMIN` authority. It also configures HTTP basic authentication so that client applications can submit encoded authentication information in their request `Authorization` headers.

The only real problem with securing Actuator this way is that the path to the endpoints is hardcoded as /actuator/**. If this were to change because of a change to the `management.endpoints.web.base-path` property, it would no longer work. To help with this, Spring Boot also provides `EndpointRequest`—a request matcher class that makes this even easier and less dependent on a given `String` path. Using `EndpointRequest`, you can apply the same security requirements for Actuator endpoints without hardcoding the /actuator/** path, as shown here:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
  http
    .requestMatcher(EndpointRequest.toAnyEndpoint())
      .authorizeRequests()
```

```
        .anyRequest().hasRole("ADMIN")
    .and()
    .httpBasic();
}
```

The `EndpointRequest.toAnyEndpoint()` method returns a request matcher that matches any Actuator endpoint. If you'd like to exclude some of the endpoints from the request matcher, you can call `excluding()`, specifying them by name as follows:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
  http
    .requestMatcher(
        EndpointRequest.toAnyEndpoint()
                       .excluding("health", "info"))
    .authorizeRequests()
      .anyRequest().hasRole("ADMIN")
  .and()
    .httpBasic();
}
```

On the other hand, should you wish to apply security to only a handful of Actuator endpoints, you can specify those endpoints by name by calling `to()` instead of `toAnyEndpoint()`, like this:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
  http
    .requestMatcher(EndpointRequest.to(
            "beans", "threaddump", "loggers"))
    .authorizeRequests()
      .anyRequest().hasRole("ADMIN")
  .and()
    .httpBasic();
}
```

This limits Actuator security to only the /beans, /threaddump, and /loggers endpoints. All other Actuator endpoints are left wide open.

Summary

- Spring Boot Actuator provides several endpoints, both as HTTP and JMX MBeans, that let you peek into the inner workings of a Spring Boot application.
- Most Actuator endpoints are disabled by default but can be selectively exposed by setting `management.endpoints.web.exposure.include` and `management.endpoints.web.exposure.exclude`.
- Some endpoints, such as the /loggers and /env endpoints, allow for write operations to change a running application's configuration on the fly.
- Details regarding an application's build and Git commit can be exposed in the /info endpoint.

- An application's health can be influenced by a custom health indicator, tracking the health of an externally integrated application.
- Custom application metrics can be registered through Micrometer, which affords Spring Boot applications instant integration with several popular metrics engines such as Datadog, New Relic, and Prometheus.
- Actuator web endpoints can be secured using Spring Security, much like any other endpoint in a Spring application.

16 Administering Spring

This chapter covers

- Setting up Spring Boot Admin
- Registering client applications
- Working with Actuator endpoints
- Securing the Admin server

A picture is worth a thousand words (or so they say), and for many application users, a user-friendly web application is worth a thousand API calls. Don't get me wrong, I'm a command-line junkie and a big fan of using `curl` and HTTPie to consume REST APIs. But sometimes, manually typing the command line to invoke a REST endpoint and then visually inspecting the results can be less efficient than simply clicking a link and reading the results in a web browser.

In the previous chapter, we explored all of the HTTP endpoints exposed by the Spring Boot Actuator. As HTTP endpoints that return JSON responses, there's no limit to how those can be used. In this chapter, we'll see how to put a frontend user interface (UI) on top of the Actuator to make it easier to use, as well as capture live data that would be difficult to consume from Actuator directly.

16.1 Using Spring Boot Admin

I've been asked several times if it'd make sense and, if so, how hard it'd be to develop a web application that consumes Actuator endpoints and serves them up in an easy-to-view UI. I respond that it's just a REST API, and, therefore, anything is possible. But why bother creating your own UI for the Actuator when the good folks at codecentric AG (https://www.codecentric.de/), a software and consulting company based in Germany, have already done the work for you?

Spring Boot Admin is an administrative frontend web application that makes Actuator endpoints more consumable by humans. It's split into two primary components: the Spring Boot Admin server and its clients. The Admin server collects and displays Actuator data that's fed to it from one or more Spring Boot applications, which are identified as Spring Boot Admin clients, as illustrated in figure 16.1.

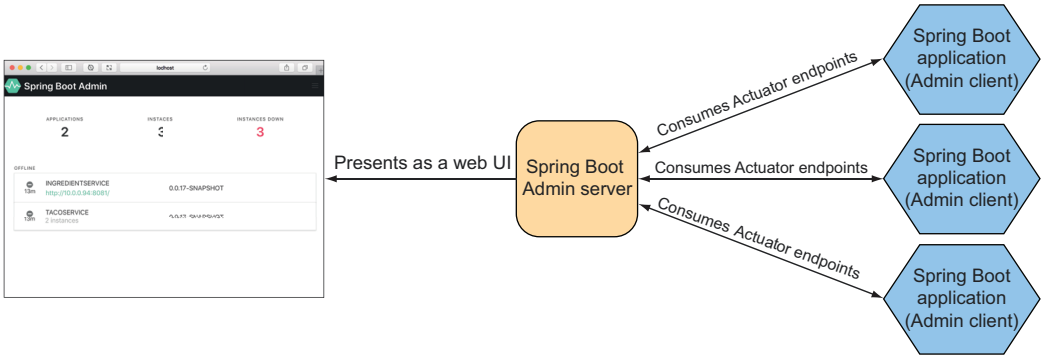

Figure 16.1 The Spring Boot Admin server consumes Actuator endpoints from one or more Spring Boot applications and presents the data in a web-based UI.

You'll need to register each of your applications with the Spring Boot Admin server, including the Taco Cloud application. But first, you'll set up the Spring Boot Admin server to receive each client's Actuator information.

16.1.1 Creating an Admin server

To enable the Admin server, you'll first need to create a new Spring Boot application and add the Admin server dependency to the project's build. The Admin server is generally used as a standalone application, separate from any other application. Therefore, the easiest way to get started is to use the Spring Boot Initializr to create a new Spring Boot project and select the check box labeled Spring Boot Admin (Server). This results in the following dependency being included in the `<dependencies>` block:

```
<dependency>
  <groupId>de.codecentric</groupId>
  <artifactId>spring-boot-admin-starter-server</artifactId>
</dependency>
```

Next, you'll need to enable the Admin server by annotating the main configuration class with `@EnableAdminServer` as shown here:

```
package tacos.admin;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import de.codecentric.boot.admin.server.config.EnableAdminServer;

@EnableAdminServer
@SpringBootApplication
public class AdminServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(AdminServerApplication.class, args);
    }

}
```

Finally, because the Admin server won't be the only application running locally as it's developed, you should set it to listen in on a unique port, but one you can easily access (not port 0, for example). Here, I've chosen port 9090 as the port for the Spring Boot Admin server:

```
server:
  port: 9090
```

Now your Admin server is ready. If you were to fire it up at this point and navigate to http://localhost:9090 in your web browser, you'd see something like what's shown in figure 16.2.

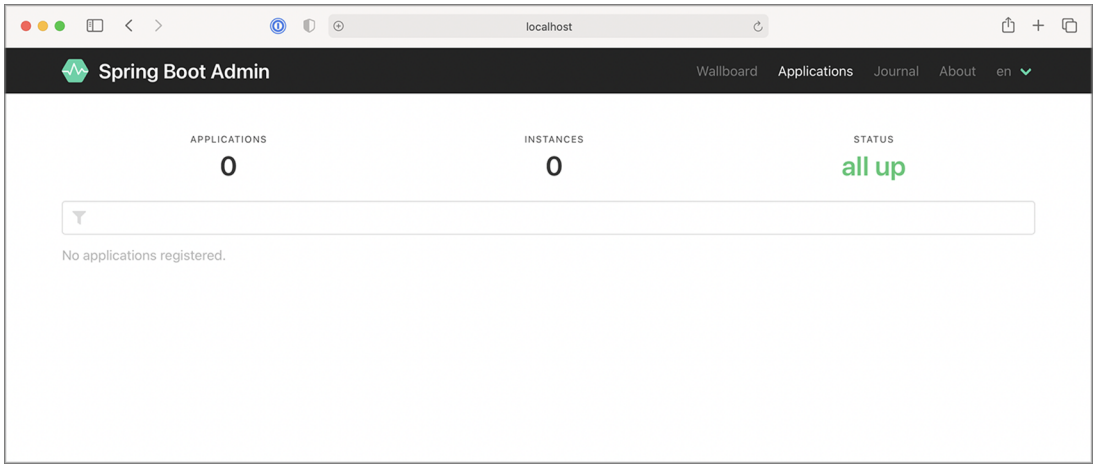

Figure 16.2 A newly created server displayed in the Spring Boot Admin UI. No applications are registered yet.

As you can see, the Spring Boot Admin shows that zero instances of zero applications are all up. But that's meaningless information when you consider the message below those counts that states No Applications Registered. For the Admin server to be useful, you'll need to register some applications with it.

16.1.2 Registering Admin clients

Because the Admin server is an application separate from other Spring Boot application(s) for which it presents Actuator data, you must somehow make the Admin server aware of the applications it should display. Two ways to register Spring Boot Admin clients with the Admin server follow:

- Each application explicitly registers itself with the Admin server.
- The Admin server discovers applications through the Eureka service registry.

We'll focus on how to configure individual Boot applications as Spring Boot Admin clients so that they can register themselves with the Admin server. For more information about working with Eureka, see the Spring Cloud documentation at https://docs.spring.io/spring-cloud-netflix/docs/current/reference/html/ or *Spring Microservices in Action, 2nd Edition*, by John Carnell and Illary Huaylupo Sánchez.

For a Spring Boot application to register itself as a client of the Admin server, you must include the Spring Boot Admin client starter in its build. You can easily add this dependency to your build by selecting the check box labeled Spring Boot Admin (Client) in the Initializr, or you can set the following `<dependency>` for a Maven-built Spring Boot application:

```
<dependency>
  <groupId>de.codecentric</groupId>
  <artifactId>spring-boot-admin-starter-client</artifactId>
</dependency>
```

With the client-side library in place, you'll also need to configure the location of the Admin server so that the client can register itself. To do that, you'll set the `spring.boot.admin.client.url` property to the root URL of the Admin server like so:

```
spring:
  boot:
    admin:
      client:
        url: http://localhost:9090
```

Notice that the `spring.application.name` property is also set. This property is used by several Spring projects to identify an application. In this case, it is the name that will be given to the Admin server to use as a label anywhere information about the application appears in the Admin server.

Although there isn't much information about the Taco Cloud application shown in figure 16.3, it does show the application's uptime, whether the Spring Boot Maven plugin has the `build-info` goal configured (as we discussed in section 15.3.1), and the build version. Rest assured that you'll see plenty of other runtime details after you click the application in the Admin server.

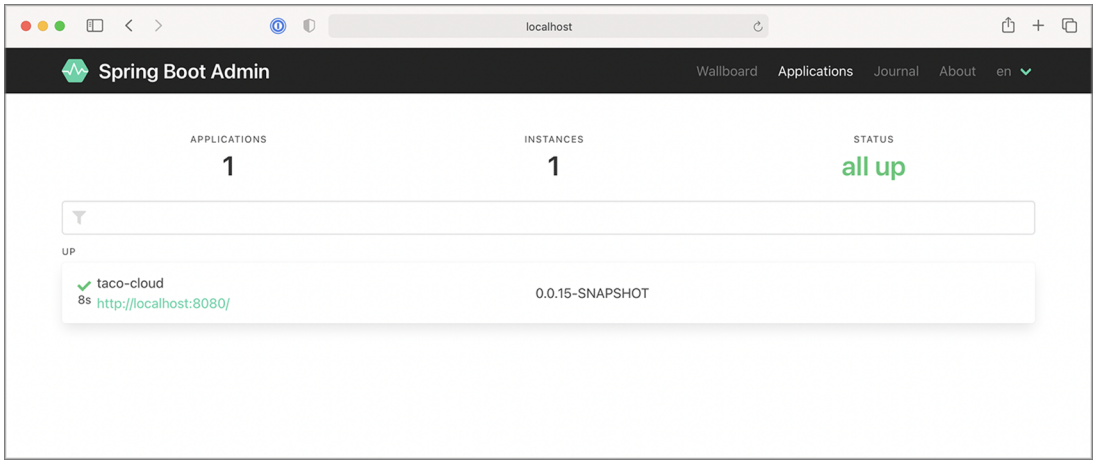

Figure 16.3 The Spring Boot Admin UI displays a single registered application.

Now that you have the Taco Cloud application registered with the Admin server, let's see what the Admin server has to offer.

16.2 Exploring the Admin server

Once you've registered all of the Spring Boot applications as Admin server clients, the Admin server makes a wealth of information available for seeing what's going on inside each application, including the following:

- General health and information
- Any metrics published through Micrometer and the /metrics endpoint
- Environment properties
- Logging levels for packages and classes

In fact, almost anything that the Actuator exposes can be viewed in the Admin server, albeit in a much more human-friendly format. This includes graphs and filters to help distill the information. The amount of information presented in the Admin server is far richer than the space we'll have in this chapter to cover it in detail. But let me use the rest of this section to share a few of the highlights of the Admin server.

16.2.1 Viewing general application health and information

As discussed in section 15, some of the most basic information provided by the Actuator is health and general application information via the /health and /info endpoints. The Admin server displays that information under the Details menu item as shown in figure 16.4.

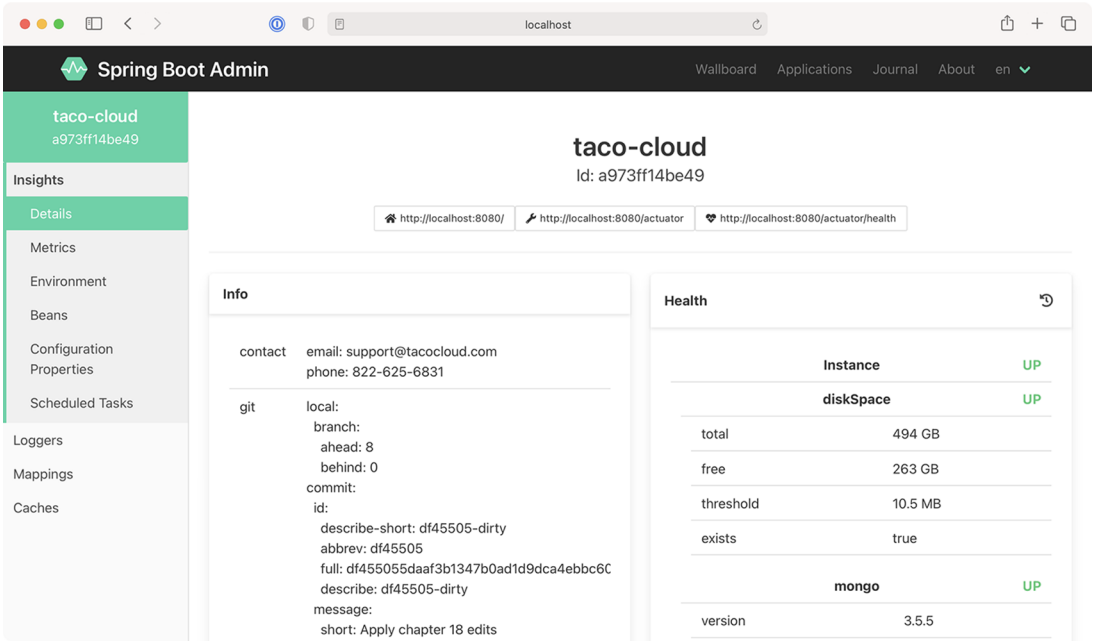

Figure 16.4 The Details screen of the Spring Boot Admin UI displays general health and information about an application.

If you scroll past the Health and Info sections in the Details screen, you'll find useful statistics from the application's JVM, including graphs displaying memory, thread, and processor usage (see figure 16.5).

The information displayed in the graphs, as well as the metrics under Processes and Garbage Collection Pauses, can provide useful insights into how your application uses JVM resources.

16.2.2 Watching key metrics

The information presented by the /metrics endpoint is perhaps the least human-readable of all of the Actuator's endpoints. But the Admin server makes it easy for us mere mortals to consume the metrics produced in an application with its UI under the Metrics menu item.

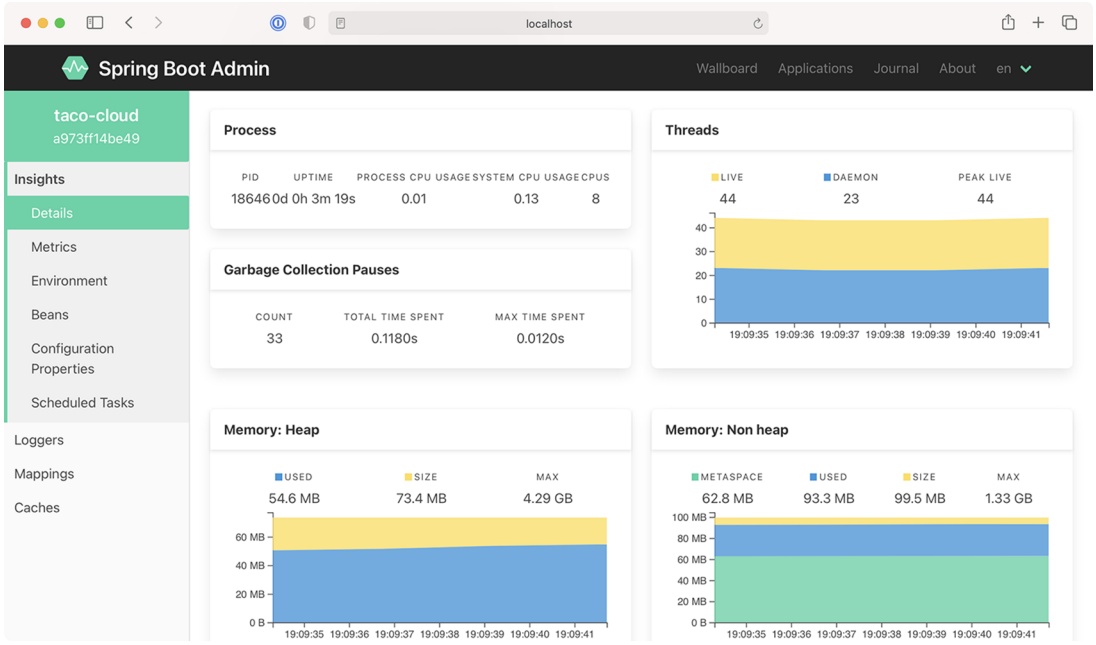

Figure 16.5 As you scroll down on the Details screen, you can view additional JVM internal information, including processor, thread, and memory statistics.

Initially, the Metrics screen doesn't display any metrics whatsoever. But the form at the top lets you set up one or more watches on any metrics you want to keep an eye on.

In figure 16.6, I've set up two watches on metrics under the http.server.requests category. The first reports metrics anytime an HTTP `GET` request is received and the return status is 200 (OK). The second reports metrics for any request that results in an HTTP 404 (NOT FOUND) response.

What's nice about these metrics (and, in fact, almost anything displayed in the Admin server) is that they show live data—they'll automatically update without the need to refresh the page.

16.2.3 Examining environment properties

The Actuator's /env endpoint returns all environment properties available to a Spring Boot application from all of its property sources. And although the JSON response from the endpoint isn't all that difficult to read, the Admin server presents it in a much more aesthetically pleasing form under the Environment menu item, shown in figure 16.7.

Because there can be hundreds of properties, you can filter the list of available properties by either property name or value. Figure 16.7 shows properties filtered by those whose name and/or values contain the text `"spring."`. The Admin server also

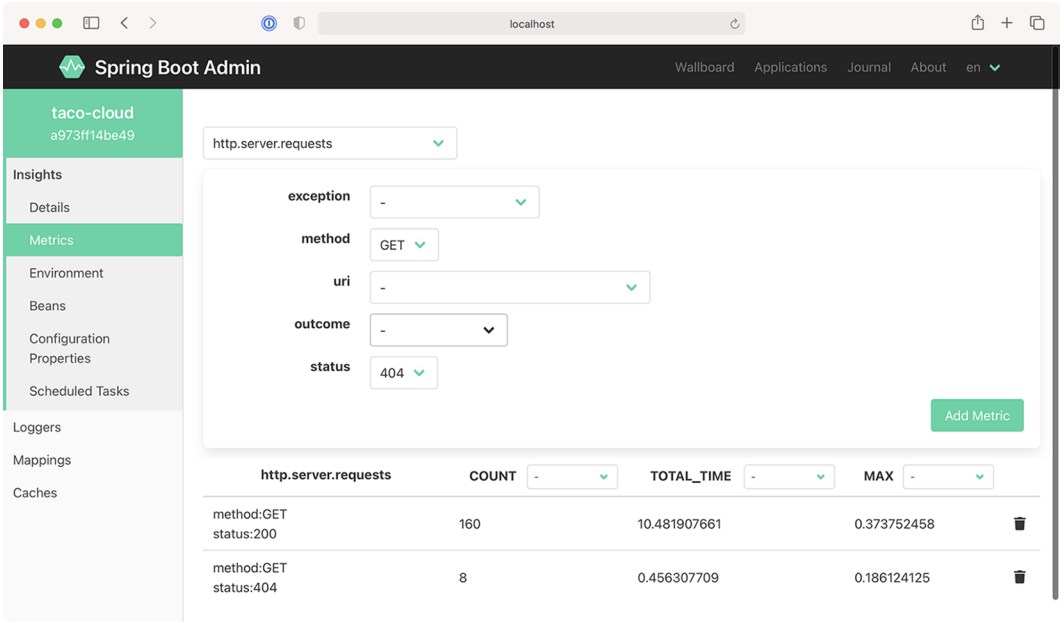

Figure 16.6 On the Metrics screen, you can set up watches on any metrics published through the application's /metrics endpoint.

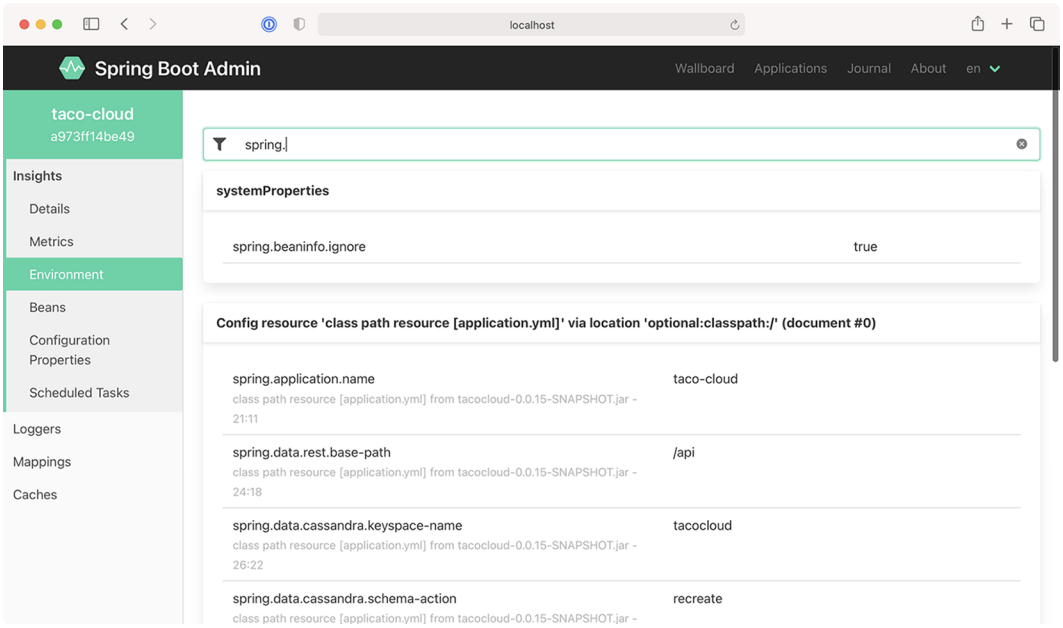

Figure 16.7 The Environment screen displays environment properties and includes options to override and filter those values.

allows you to set or override environment properties using the form under the Environment Manager header.

16.2.4 Viewing and setting logging levels

The Actuator's /loggers endpoint is helpful in understanding and overriding logging levels in a running application. The Admin server's Loggers screen adds an easy-to-use UI on top of the /loggers endpoint to make simple work of managing logging in an application. Figure 16.8 shows the list of loggers filtered by the name org.springframework.boot.

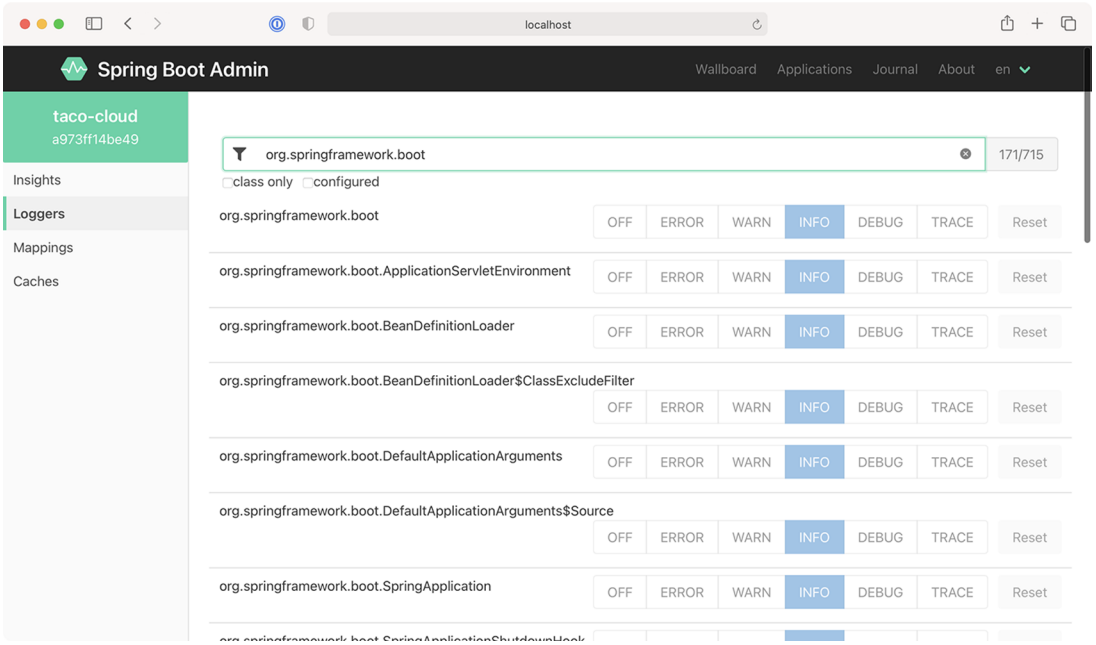

Figure 16.8 The Loggers screen displays logging levels for packages and classes in the application and lets you override those levels.

By default, the Admin server displays logging levels for all packages and classes. Those can be filtered by name (for classes only) and/or logging levels that are explicitly configured versus inherited from the root logger.

16.3 Securing the Admin server

As we discussed in the previous chapter, the information exposed by the Actuator's endpoints isn't intended for general consumption. They contain information that exposes details about an application that only an application administrator should

see. Moreover, some of the endpoints allow changes that certainly shouldn't be exposed to just anyone.

Just as security is important to the Actuator, it's also important to the Admin server. What's more, if the Actuator endpoints require authentication, then the Admin server needs to know the credentials to be able to access those endpoints. Let's see how to add a little security to the Admin server. We'll start by requiring authentication.

16.3.1 Enabling login in the Admin server

It's probably a good idea to add security to the Admin server because it's not secured by default. Because the Admin server is a Spring Boot application, you can secure it using Spring Security just like you would any other Spring Boot application. And just as you would with any application secured by Spring Security, you're free to decide which security scheme fits your needs best.

At a minimum, you can add the Spring Boot security starter to the Admin server's build by checking the Security checkbox in the Initializr or by adding the following `<dependency>` to the project's pom.xml file:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Then, so that you don't have to keep looking at the Admin server's logs for the randomly generated password, you can configure a simple administrative username and password in application.yml like so:

```
spring:
  security:
    user:
      name: admin
      password: 53cr3t
```

Now when the Admin server is loaded in the browser, you'll be prompted for a username and password with Spring Security's default login form. As in the code snippet, entering `admin` and `53cr3t` will get you in.

By default, Spring Security will enable CSRF on the Spring Boot Admin server, which will prevent client applications from registering with the Admin Server. Therefore, we will need a small bit of security configuration to disable CSRF, as shown here:

```
package tacos.admin;

import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.reactive
     .EnableWebFluxSecurity;
import org.springframework.security.config.web.server.ServerHttpSecurity;
import org.springframework.security.web.server.SecurityWebFilterChain;
```

```
@EnableWebFluxSecurity
public class SecurityConfig {

    @Bean
    public SecurityWebFilterChain filterChain(ServerHttpSecurity http) throws
     Exception {
      return http
              .csrf()
                  .disable()
              .build();
    }

}
```

Of course, this security configuration is extremely basic. I recommend that you consult chapter 5 for ways of configuring Spring Security for a richer security scheme around the Admin server.

16.3.2 Authenticating with the Actuator

In section 15.4, we discussed how to secure Actuator endpoints with HTTP Basic authentication. By doing so, you'll be able to keep out everyone who doesn't know the username and password you assigned to the Actuator endpoints. Unfortunately, that also means that the Admin server won't be able to consume Actuator endpoints unless it provides the username and password. But how will the Admin server get those credentials?

If the application registers directly with the Admin server, then it can send its credentials to the server at registration time. You'll need to configure a few properties to enable that.

The `spring.boot.admin.client.username` and `spring.boot.admin.client.password` properties specify the credentials that the Admin server can use to access an application's Actuator endpoints. The following snippet from application.yml shows how you might set those properties:

```
spring:
  boot:
    admin:
      client:
        url: http://localhost:9090
        username: admin
        password: 53cr3t
```

The username and password properties must be set in each application that registers itself with the Admin server. The values given must match the username and password that's required in an HTTP Basic authentication header to the Actuator endpoints. In this example, they're set to `admin` and `password`, which are the credentials configured to access the Actuator endpoints.

Summary

- The Spring Boot Admin server consumes the Actuator endpoints from one or more Spring Boot applications and presents the data in a user-friendly web application.
- Spring Boot applications can either register themselves as clients to the Admin server or the Admin server can discover them through Eureka.
- Unlike the Actuator endpoints that capture a snapshot of an application's state, the Admin server is able to display a live view into the inner workings of an application.
- The Admin server makes it easy to filter Actuator results and, in some cases, display data visually in a graph.
- Because it's a Spring Boot application, the Admin server can be secured by any means available through Spring Security.

17 Monitoring Spring with JMX

This chapter covers

- Working with Actuator endpoint MBeans
- Exposing Spring beans as MBeans
- Publishing notifications

For over a decade and a half, Java Management Extensions (JMX) has been the standard means of monitoring and managing Java applications. By exposing managed components known as MBeans (managed beans), an external JMX client can manage an application by invoking operations, inspecting properties, and monitoring events from MBeans.

We'll start exploring Spring and JMX by looking at how Actuator endpoints are exposed as MBeans.

17.1 Working with Actuator MBeans

By default, all Actuator endpoints are exposed as MBeans. But, starting with Spring Boot 2.2, JMX itself is disabled by default. To enable JMX in your Spring Boot application, you can set `spring.jmx.enabled` to `true`. In application.yml, this would look like this:

```
spring:
  jmx:
    enabled: true
```

With that property set, Spring support for JMX is enabled. And with it, the Actuator endpoints are all exposed as MBeans. You can use any JMX client you wish to connect with Actuator endpoint MBeans. Using JConsole, which comes with the Java Development Kit, you'll find Actuator MBeans listed under the `org.springframework.boot` domain, as shown in figure 17.1.

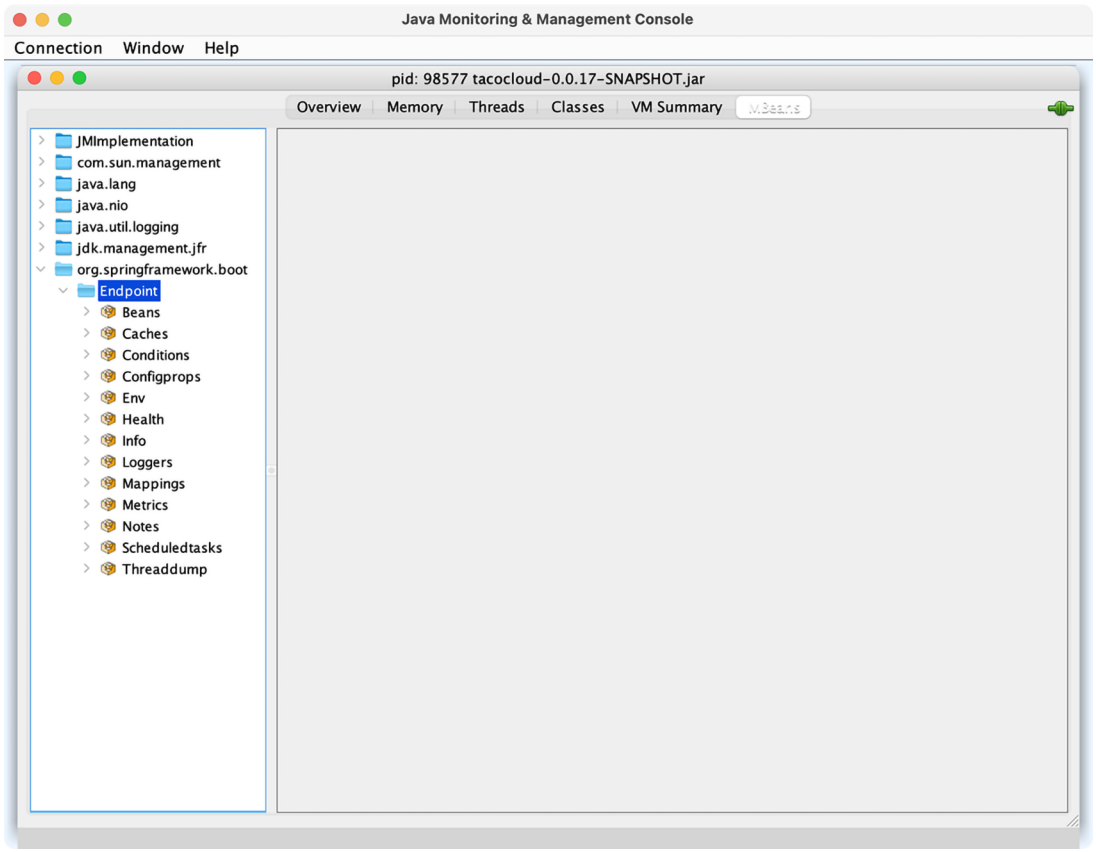

Figure 17.1 Actuator endpoints are automatically exposed as JMX MBeans.

One thing that's nice about Actuator MBean endpoints is that they're all exposed by default. There's no need to explicitly include any of them, as you had to do with HTTP. You can, however, choose to narrow down the choices by setting `management.endpoints.jmx.exposure.include` and `management.endpoints.jmx.exposure.exclude`. For example, to limit Actuator endpoint MBeans to only the /health,

/info, /bean, and /conditions endpoints, set `management.endpoints.jmx.exposure.include` like this:

```
management:
  endpoints:
    jmx:
      exposure:
        include: health,info,bean,conditions
```

Or, if there are only a few you want to exclude, you can set `management.endpoints.jmx.exposure.exclude` like this:

```
management:
  endpoints:
    jmx:
      exposure:
        exclude: env,metrics
```

Here, you use `management.endpoints.jmx.exposure.exclude` to exclude the /env and /metrics endpoints. All other Actuator endpoints will still be exposed as MBeans.

To invoke the managed operations on one of the Actuator MBeans in JConsole, expand the endpoint MBean in the left-hand tree, and then select the desired operation under Operations.

For example, if you'd like to inspect the logging levels for the `tacos.ingredients` package, expand the `Loggers` MBean and click on the operation named `loggerLevels`, as shown in figure 17.2. In the form at the top right, fill in the Name field with the package name (`org.springframework.web`, for example), and then click the `loggerLevels` button.

After you click the `loggerLevels` button, a dialog box will pop up, showing you the response from the /loggers endpoint MBean. It might look a little like figure 17.3.

Although the JConsole UI is a bit clumsy to work with, you should be able to get the hang of it and use it to explore any Actuator endpoint in much the same way. If you don't like JConsole, that's fine—there are plenty of other JMX clients to choose from.

17.2 Creating your own MBeans

Spring makes it easy to expose any bean you want as a JMX MBean. All you must do is annotate the bean class with `@ManagedResource` and then annotate any methods or properties with `@ManagedOperation` or `@ManagedAttribute`. Spring will take care of the rest.

For example, suppose you want to provide an MBean that tracks how many tacos have been ordered through Taco Cloud. You can define a service bean that keeps a running count of how many tacos have been created. The following listing shows what such a service might look like.

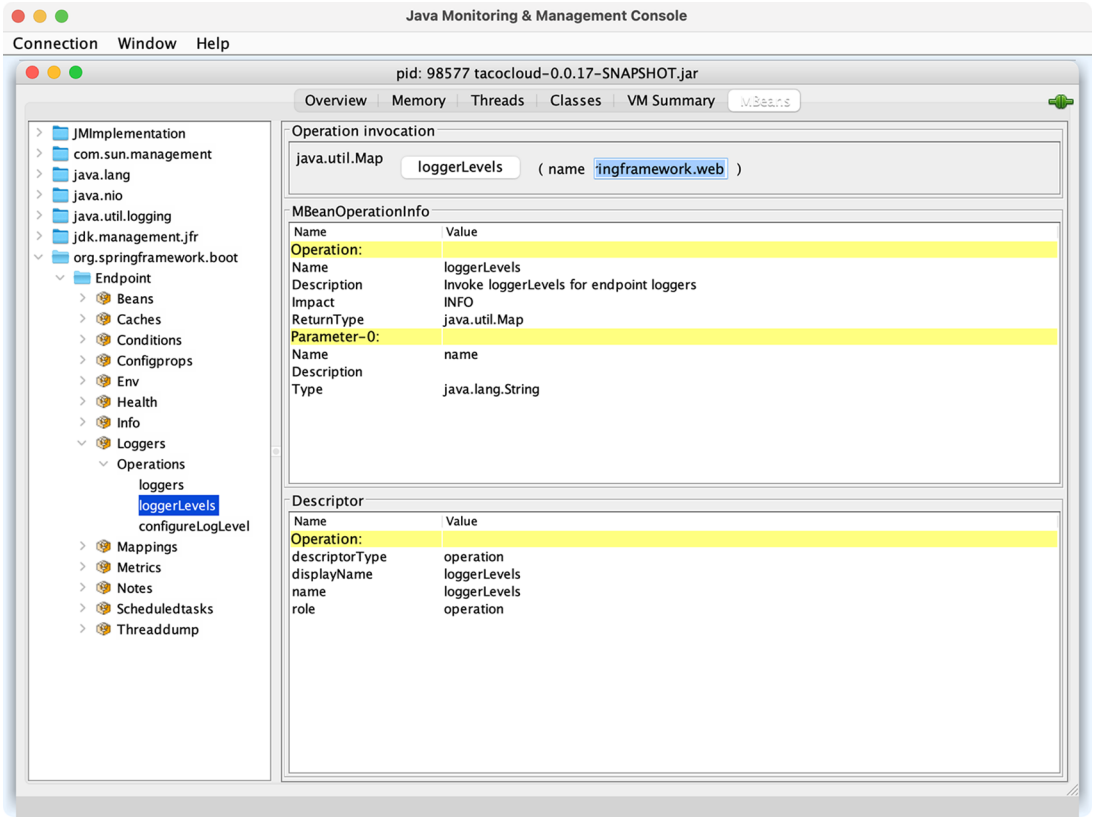

Figure 17.2 Using JConsole to display logging levels from a Spring Boot application

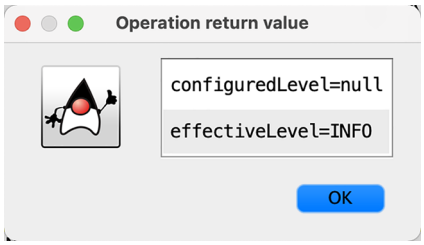

Figure 17.3 Logging levels from the /loggers endpoint MBean displayed in JConsole

Listing 17.1 An MBean that counts how many tacos have been created

```
package tacos.jmx;

import java.util.concurrent.atomic.AtomicLong;
import org.springframework.data.rest.core.event.AbstractRepositoryEventListener;
import org.springframework.jmx.export.annotation.ManagedAttribute;
import org.springframework.jmx.export.annotation.ManagedOperation;
```

```
import org.springframework.jmx.export.annotation.ManagedResource;
import org.springframework.stereotype.Service;
import tacos.Taco;
import tacos.data.TacoRepository;

@Service
@ManagedResource
public class TacoCounter
       extends AbstractRepositoryEventListener<Taco> {

  private AtomicLong counter;
  public TacoCounter(TacoRepository tacoRepo) {
    tacoRepo
        .count()
        .subscribe(initialCount -> {
            this.counter = new AtomicLong(initialCount);
        });
  }

  @Override
  protected void onAfterCreate(Taco entity) {
    counter.incrementAndGet();
  }

  @ManagedAttribute
  public long getTacoCount() {
    return counter.get();
  }

  @ManagedOperation
  public long increment(long delta) {
    return counter.addAndGet(delta);
  }

}
```

The `TacoCounter` class is annotated with `@Service` so that it will be picked up by component scanning and an instance will be registered as a bean in the Spring application context. But it's also annotated with `@ManagedResource` to indicate that this bean should also be an MBean. As an MBean, it will expose one attribute and one operation. The `getTacoCount()` method is annotated with `@ManagedAttribute` so that it will be exposed as an MBean attribute, whereas the `increment()` method is annotated with `@ManagedOperation`, exposing it as an MBean operation. Figure 17.4 shows how the `TacoCounter` MBean appears in JConsole.

`TacoCounter` has another trick up its sleeve, although it has nothing to do with JMX. Because it extends `AbstractRepositoryEventListener`, it will be notified of any persistence events when a `Taco` is saved through `TacoRepository`. In this particular case, the `onAfterCreate()` method will be invoked anytime a new `Taco` object is created and saved, and it will increment the counter by one. But `AbstractRepositoryEventListener` also offers several methods for handling events both before and after objects are created, saved, or deleted.

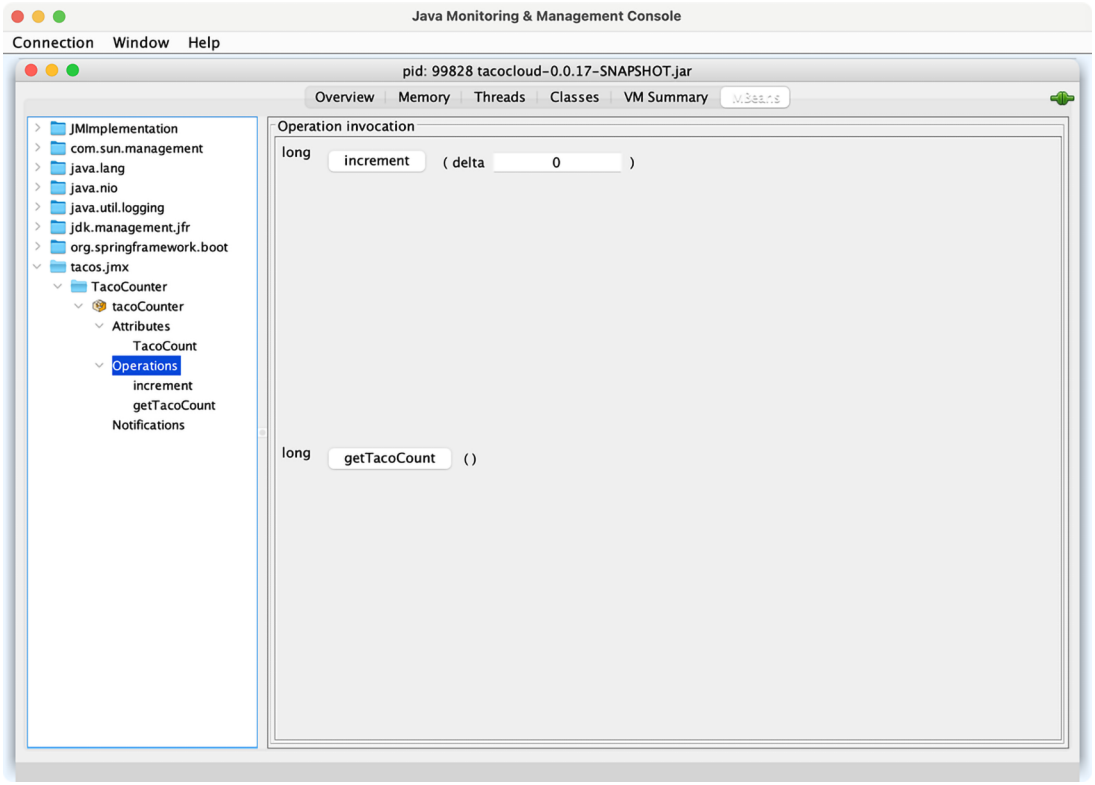

Figure 17.4 `TacoCounter`'s operations and attributes as seen in JConsole

Working with MBean operations and attributes is largely a pull operation. That is, even if the value of an MBean attribute changes, you won't know until you view the attribute through a JMX client. Let's turn the tables and see how you can push notifications from an MBean to a JMX client.

17.3 Sending notifications

MBeans can push notifications to interested JMX clients with Spring's `NotificationPublisher`. `NotificationPublisher` has a single `sendNotification()` method that, when given a `Notification` object, publishes the notification to any JMX clients that have subscribed to the MBean.

For an MBean to be able to publish notifications, it must implement the `NotificationPublisherAware` interface, which requires that a `setNotificationPublisher()` method be implemented. For example, suppose you want to publish a notification for every 100 tacos that are created. You can change the `TacoCounter` class so that it implements `NotificationPublisherAware` and uses the injected `NotificationPublisher` to send notifications for every 100 tacos that are created.

The following listing shows the changes that must be made to `TacoCounter` to enable such notifications.

Listing 17.2 Sending notifications for every 100 tacos

```
package tacos.jmx;

import java.util.concurrent.atomic.AtomicLong;
import org.springframework.data.rest.core.event.AbstractRepositoryEventListener;
import org.springframework.jmx.export.annotation.ManagedAttribute;
import org.springframework.jmx.export.annotation.ManagedOperation;
import org.springframework.jmx.export.annotation.ManagedResource;
import org.springframework.stereotype.Service;

import org.springframework.jmx.export.notification.NotificationPublisher;
import org.springframework.jmx.export.notification.NotificationPublisherAware;
import javax.management.Notification;

import tacos.Taco;
import tacos.data.TacoRepository;

@Service
@ManagedResource
public class TacoCounter
       extends AbstractRepositoryEventListener<Taco>
       implements NotificationPublisherAware {


  private AtomicLong counter;
  private NotificationPublisher np;

  @Override
  public void setNotificationPublisher(NotificationPublisher np) {
    this.np = np;
  }

  ...

  @ManagedOperation
  public long increment(long delta) {
    long before = counter.get();
    long after = counter.addAndGet(delta);
    if ((after / 100) > (before / 100)) {
      Notification notification = new Notification(
          "taco.count", this,
          before, after + "th taco created!");
      np.sendNotification(notification);
    }
    return after;
  }

}
```

In the JMX client, you'll need to subscribe to the `TacoCounter` MBean to receive notifications. Then, as tacos are created, the client will receive notifications for each century count. Figure 17.5 shows how the notifications may appear in JConsole.

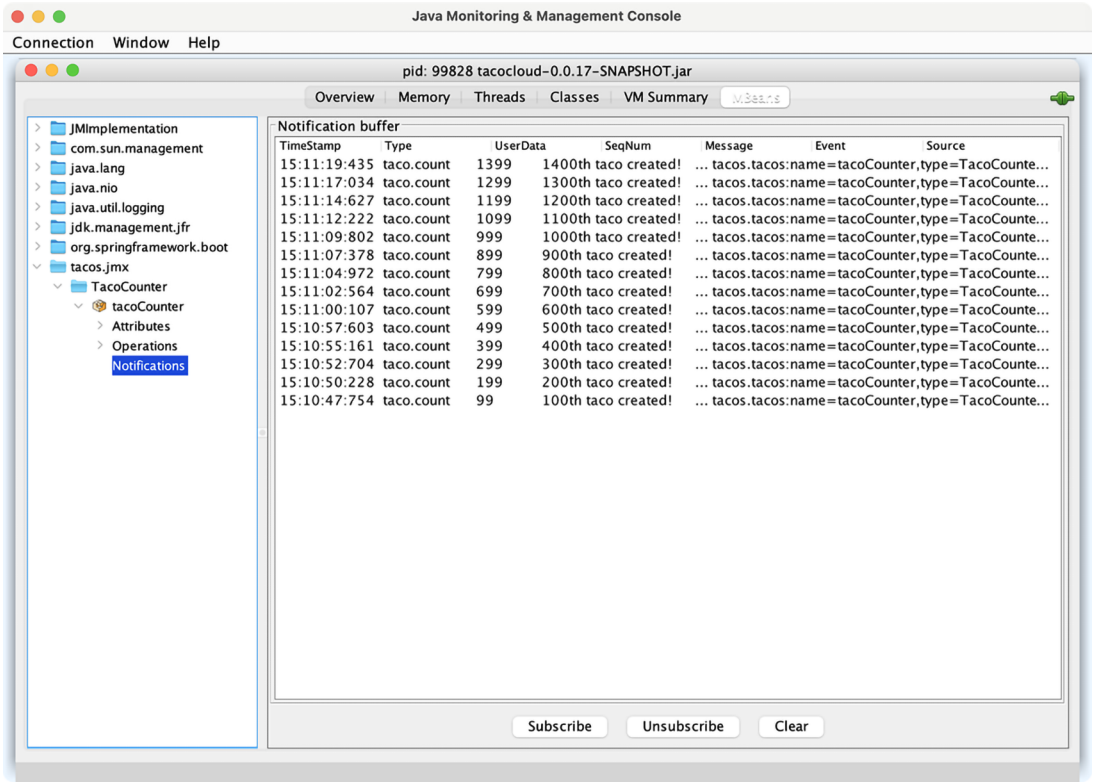

Figure 17.5 JConsole, subscribed to the `TacoCounter` MBean, receives a notification for every 100 tacos that are created.

Notifications are a great way for an application to actively send data and alerts to a monitoring client without requiring the client to poll managed attributes or invoke managed operations.

Summary

- Most Actuator endpoints are available as MBeans that can be consumed using any JMX client.
- Spring automatically enables JMX for monitoring beans in the Spring application context.
- Spring beans can be exposed as MBeans by annotating them with `@ManagedResource`. Their methods and properties can be exposed as managed operations and attributes by annotating the bean class with `@ManagedOperation` and `@ManagedAttribute`.
- Spring beans can publish notifications to JMX clients using `NotificationPublisher`.

18 Deploying Spring

This chapter covers

- Building Spring applications as either WAR or JAR files
- Building Spring applications as container images
- Deploying Spring applications in Kubernetes

Think of your favorite action movie. Now imagine going to see that movie in the theater and being taken on a thrilling audiovisual ride with high-speed chases, explosions, and battles, only to have it come to a sudden halt before the good guys take down the bad guys. Instead of seeing the movie's conflict resolved, when the theater lights come on, everyone is ushered out the door. Although the lead-up was exciting, it's the climax of the movie that's important. Without it, it's action for action's sake.

Now imagine developing applications and putting a lot of effort and creativity into solving the business problem, but then never deploying the application for others to use and enjoy. Sure, most applications we write don't involve car chases or explosions (at least I hope not), but there's a certain rush you get along the way. Not every line of code you write is destined for production, but it'd be a big let-down if none of it ever was deployed.

Up to this point, we've focused on using the features of Spring Boot that help us develop an application. There have been some exciting steps along the way, but it's all for nothing if you don't cross the finish line and deploy the application.

In this chapter, we're going to step beyond developing applications with Spring Boot and look at how to deploy those applications. Although this may seem obvious for anyone who has ever deployed a Java-based application, Spring Boot and related Spring projects have some features you can draw on that make deploying Spring Boot applications unique.

In fact, unlike most Java web applications, which are typically deployed to an application server as WAR files, Spring Boot offers several deployment options. Before we look at how to deploy a Spring Boot application, let's consider all the options and choose a few that suit your needs best.

18.1 Weighing deployment options

You can build and run Spring Boot applications in several ways, including the following:

- Running the application directly in the IDE with either Spring Tool Suite or IntelliJ IDEA
- Running the application from the command line using the Maven `spring-boot:run` goal or Gradle `bootRun` task
- Using Maven or Gradle to produce an executable JAR file that can be run at the command line or be deployed in the cloud
- Using Maven or Gradle to produce a WAR file that can be deployed to a traditional Java application server
- Using Maven or Gradle to produce a container image that can be deployed anywhere that containers are supported, including Kubernetes environments.

Any of these choices is suitable for running the application while you're still developing it. But what about when you're ready to deploy the application into a production or other nondevelopment environment?

Although running an application from the IDE or via Maven or Gradle isn't considered a production-ready option, executable JAR files and traditional Java WAR files are certainly valid options for deploying applications to a production environment. Given the options of deploying a WAR file, a JAR file, or a container image, how do you choose? In general, the choice comes down to whether you plan to deploy your application to a traditional Java application server or a cloud platform, as described here:

- *Deploying to a Platform as a Service (PaaS) cloud*—If you're planning to deploy your application to a PaaS cloud platform such as Cloud Foundry (https://www.cloudfoundry.org/), then an executable JAR file is a fine choice. Even if the cloud platform supports WAR deployment, the JAR file format is much simpler than the WAR format, which is designed for application server deployment.

- *Deploying to Java application servers*—If you must deploy your application to Tomcat, WebSphere, WebLogic, or any other traditional Java application server, you really have no choice but to build your application as a WAR file.
- *Deploying to Kubernetes*—Modern cloud platforms are increasingly based on Kubernetes (https://kubernetes.io/). When deploying to Kubernetes, which is itself a container-orchestration system, the obvious choice is to build your application into a container image.

In this chapter, we'll focus on the following three deployment scenarios:

- Building a Spring Boot application as an executable JAR file, which can possibly be pushed to a PaaS platform
- Deploying a Spring Boot application as a WAR file to a Java application server such as Tomcat
- Packaging a Spring Boot application as a Docker container image for deployment to any platform that supports Docker deployments

To get started, let's take a look at what is perhaps the most common way of building a Spring Boot application: as an executable JAR file.

18.2 Building executable JAR files

Building a Spring application into an executable JAR file is rather straightforward. Assuming that you chose JAR packaging when initializing your project, then you should be able to produce an executable JAR file with the following Maven command:

```
$ mvnw package
```

After a successful build, the resulting JAR file will be placed into the `target` directory with a name and version based on the `<artifactId>` and `<version>` entries in the project's pom.xml file (e.g., tacocloud-0.0.19-SNAPSHOT.jar).

Or, if you're using Gradle, then this will do the trick:

```
$ gradlew build
```

For Gradle builds, the resulting JAR will be found in the build/libs directory. The name of the JAR file will be based on the `rootProject.name` property in the settings .gradle file along with the `version` property in build.gradle.

Once you have the executable JAR file, you can run it with `java -jar` like this:

```
$ java -jar tacocloud-0.0.19-SNAPSHOT.jar
```

The application will run and, assuming it is a web application, start up an embedded server (Netty or Tomcat, depending on whether or not the project is a reactive web project) and start listening for requests on the configured `server.port` (8080 by default).

That's great for running the application locally. But how can you deploy an executable JAR file?

That really depends on where you'll be deploying the application. But if you are deploying to a Cloud Foundry foundation, you can push the JAR file using the `cf` command-line tool as follows:

```
$ cf push tacocloud -p target/tacocloud-0.0.19-SNAPSHOT.jar
```

The first argument to `cf push` is the name given to the application in Cloud Foundry. This name is used to reference the application in Cloud Foundry and the `cf` CLI, as well as used as a subdomain at which the application is hosted. For example, if the application domain for your Cloud Foundry foundation is cf.myorg.com, then the Taco Cloud application will be available at https://tacocloud.cf.myorg.com.

Another way to deploy executable JAR files is to package them in a Docker container and run them in Docker or Kubernetes. Let's see how to do that next.

18.3 Building container images

Docker (https://www.docker.com/) has become the de facto standard for distributing applications of all kinds for deployment in the cloud. Many different cloud environments, including AWS, Microsoft Azure, and Google Cloud Platform (to name a few) accept Docker containers for deploying applications.

The idea of containerized applications, such as those created with Docker, draws analogies from real-world intermodal containers that are used to ship items all over the world. Intermodal containers all have a standard size and format, regardless of their contents. Because of that, intermodal containers are easily stacked on ships, carried on trains, or pulled by trucks. In a similar way, containerized applications share a common container format that can be deployed and run anywhere, regardless of the application inside.

The most basic way to create an image from your Spring Boot application is to use the `docker build` command and a Dockerfile that copies the executable JAR file from the project build into the container image. The following extremely simple Dockerfile does exactly that:

```
FROM openjdk:11.0.12-jre
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

The Dockerfile describes how the container image will be created. Because it's so brief, let's examine this Dockerfile line by line:

- *Line 1*—Declares that the image we create will be based on a predefined container image that provides (among other things) the Open JDK 11 Java runtime.
- *Line 2*—Creates a variable that references all JAR files in the project's target/ directory. For most Maven builds, there should be only one JAR file in there. By using a wildcard, however, we decouple the Dockerfile definition from the JAR

file's name and version. The path to the JAR file assumes that the Dockerfile is in the root of the Maven project.

- *Line 3*—Copies the JAR file from the project's target/ directory into the container image with a generic name of app.jar.
- *Line 4*—Defines an entry point—that is, defines a command to run when a container created from this image starts—to run the JAR file with `java -jar /app.jar`.

With this Dockerfile in hand, you can create the image using the Docker command-line tool like this:

```
$ docker build . -t habuma/tacocloud:0.0.19-SNAPSHOT
```

The `.` in this command references the relative path to the location of the Dockerfile. If you are running `docker build` from a different path, replace the `.` with the path to the Dockerfile (without the filename). For example, if you are running `docker build` from the parent of the project, you will use `docker build` like this:

```
$ docker build tacocloud -t habuma/tacocloud:0.0.19-SNAPSHOT
```

The value given after the `-t` argument is the image tag, which is made up of a name and version. In this case, the image name is habuma/tacocloud and the version is 0.0.19-SNAPSHOT. If you'd like to try it out, you can use `docker run` to run this newly created image:

```
$ docker run -p8080:8080 habuma/tacocloud:0.0.19-SNAPSHOT
```

The `-p8080:8080` forwards requests to port 8080 on the host machine (e.g., your machine where you're running Docker) to the container's port 8080 (where Tomcat or Netty is listening for requests).

While building a Docker image this way is easy enough if you already have an executable JAR file handy, it's not the easiest way to create an image from a Spring Boot application. Beginning with Spring Boot 2.3.0, you can build container images without adding any special dependencies or configuration files, or editing your project in any way. That's because the Spring Boot build plugins for both Maven and Gradle support the building of container images directly. To build your Maven-built Spring project into a container image, you use the `build-image` goal from the Spring Boot Maven plugin like this:

```
$ mvnw spring-boot:build-image
```

Likewise, a Gradle-built project can be built into a container image like this:

```
$ gradlew bootBuildImage
```

This builds an image with a default tag based on the `<artifactId>` and `<version>` properties in the pom.xml file. For the Taco Cloud application, this will be something

like library/tacocloud:0.0.19-SNAPSHOT. We'll see in a moment how to specify a custom image tag.

Spring Boot's build plugins rely on Docker to create images. Therefore, you'll need to have the Docker runtime installed on the machine building the image. But once the image has been created, you can run it like this:

```
$ docker run -p8080:8080 library/tacocloud:0.0.19-SNAPSHOT
```

This runs the image and exposes the image's port 8080 (which the embedded Tomcat or Netty server is listening on) to the host machine's port 8080.

The default format of the tag is docker.io/library/ ${project.artifactId}:${project.version}, which explains why the tag began with "library." That's fine if you'll only ever be running the image locally. But you'll most likely want to push the image to an image registry such as DockerHub and will need the image to be built with a tag that references your image repository's name.

For example, suppose that your organization's repository name in DockerHub is tacocloud. In that case, you'll want the image name to be tacocloud/tacocloud:0.0.19-SNAPSHOT, effectively replacing the "library" default prefix with "tacocloud." To make that happen, you just need to specify a build property when building the image. For Maven, you'll specify the image name using the `spring-boot.build-image.imageName` JVM system property like this:

```
$ mvnw spring-boot:build-image \
    -Dspring-boot.build-image.imageName=tacocloud/tacocloud:0.0.19-SNAPSHOT
```

For a Gradle-built project, it's slightly simpler. You specify the image name using an `--imageName` parameter like this:

```
$ gradlew bootBuildImage --imageName=tacocloud/tacocloud:0.0.19-SNAPSHOT
```

Either of these ways of specifying the image name requires you to remember to do them when building the image and requires that you not make a mistake. To make things even easier, you can specify the image name as part of the build itself.

In Maven, you specify the image name as a configuration entry in the Spring Boot Maven Plugin. For example, the following snippet from the project's pom.xml file shows how to specify the image name as a `<configuration>` block:

```
<plugin>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-maven-plugin</artifactId>
 <configuration>
   <image>
     <name>tacocloud/${project.artifactId}:${project.version}</name>
   </image>
 </configuration>
</plugin>
```

Notice, that rather than hardcoding the artifact ID and version, we can leverage build variables to make those values reference what is already specified elsewhere in the build. This removes any need to manually bump the version number in the image name as a project evolves. For a Gradle-built project, the following entry in build.gradle achieves the same effect:

```
bootBuildImage {
  imageName = "habuma/${rootProject.name}:${version}"
}
```

With this configuration in place in the project build specification, you can build the image at the command line without specifying the image name, as we did earlier. At this point, you can run the image with `docker run` as before (referencing the image by its new name) or you can use `docker push` to push the image to an image registry such as DockerHub, as shown here:

```
$ docker push habuma/tacocloud:0.0.19-SNAPSHOT
```

Once the image is in an image registry, it can be pulled and run from any environment that has access to that registry. An increasingly common place to run images is in Kubernetes. Let's take a look at how to run an image in Kubernetes.

18.3.1 Deploying to Kubernetes

Kubernetes is an amazing container-orchestration platform that runs images, handles scaling containers up and down as necessary, and reconciles broken containers for increased robustness, among many other things.

Kubernetes is a powerful platform on which to deploy applications—so powerful, in fact, that there's no way we'll be able to cover it in detail in this chapter. Instead, we'll focus solely on the tasks required to deploy a Spring Boot application, built into a container image, into a Kubernetes cluster. For a more detailed understanding of Kubernetes, check out *Kubernetes in Action, 2nd Edition,* by Marko Lukša.

Kubernetes has earned a reputation of being difficult to use (perhaps unfairly), but deploying a Spring application that has been built as a container image in Kubernetes is really easy and is worth the effort given all of the benefits afforded by Kubernetes.

You'll need a Kubernetes environment into which to deploy your application. Several options are available, including Amazon's AWS EKS and the Google Kubernetes Engine (GKE). For experimentation locally, you can also run Kubernetes clusters using a variety of Kubernetes implementations such as MiniKube (https://minikube.sigs.k8s.io/docs/), MicroK8s (https://microk8s.io/), and my personal favorite, Kind (https://kind.sigs.k8s.io/).

The first thing you'll need to do is create a deployment manifest. The deployment manifest is a YAML file that describes how an image should be deployed. As a simple

example, consider the following deployment manifest that deploys the Taco Cloud image created earlier in a Kubernetes cluster:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: taco-cloud-deploy
  labels:
    app: taco-cloud
spec:
  replicas: 3
  selector:
    matchLabels:
      app: taco-cloud
  template:
    metadata:
      labels:
        app: taco-cloud
    spec:
      containers:
      - name: taco-cloud-container
        image: tacocloud/tacocloud:latest
```

This manifest can be named anything you like. But for the sake of discussion, let's assume you named it deploy.yaml and placed it in a directory named k8s at the root of the project.

Without diving into the details of how a Kubernetes deployment specification works, the key things to notice here are that our deployment is named taco-cloud-deploy and (near the bottom) is set to deploy and start a container based on the image whose name is tacocloud/tacocloud:latest. By giving "latest" as the version rather than "0.0.19-SNAPSHOT," we can know that the very latest image pushed to the container registry will be used.

Another thing to notice is that the `replicas` property is set to `3`. This tells the Kubernetes runtime that there should be three instances of the container running. If, for any reason, one of those three instances fails, then Kubernetes will automatically reconcile the problem by starting a new instance in its place. To apply the deployment, you can use the `kubectl` command-line tool like this:

```
$ kubectl apply -f deploy.yaml
```

After a moment or so, you should be able to use `kubectl get all` to see the deployment in action, including three *pods*, each one running a container instance. Here's a sample of what you might see:

```
$ kubectl get all
NAME                                      READY   STATUS    RESTARTS   AGE
pod/taco-cloud-deploy-555bd8fdb4-dln45    1/1     Running   0          20s
pod/taco-cloud-deploy-555bd8fdb4-n455b    1/1     Running   0          20s
pod/taco-cloud-deploy-555bd8fdb4-xp756    1/1     Running   0          20s
```

```
NAME                                    READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/taco-cloud-deploy       3/3     3            3           20s

NAME                                              DESIRED  CURRENT  READY  AGE
replicaset.apps/taco-cloud-deploy-555bd8fdb4      3        3        3      20s
```

The first section shows three pods, one for each instance we requested in the `replicas` property. The middle section is the deployment resource itself. And the final section is a `ReplicaSet` resource, a special resource that Kubernetes uses to remember how many replicas of the application should be maintained.

If you want to try out the application, you'll need to expose a port from one of the pods on your machine. To do that, the `kubectl port-forward` command, shown next, comes in handy:

```
$ kubectl port-forward pod/taco-cloud-deploy-555bd8fdb4-dln45 8080:8080
```

In this case, I've chosen the first of the three pods listed from `kubectl get all` and asked to forward requests from the host machine's (the machine on which the Kubernetes cluster is running) port 8080 to the pod's port 8080. With that in place, you should be able to point your browser at http://localhost:8080 to see the Taco Cloud application running on the specified pod.

18.3.2 Enabling graceful shutdown

We have several ways in which to make Spring applications Kubernetes friendly, but the two most essential things you'll want to do are to enable graceful shutdown as well as liveness and readiness probes.

At any time, Kubernetes may decide to shut down one or more of the pods that your application is running in. That may be because it senses a problem, or it might be because someone has explicitly requested that the pod be shut down or restarted. Whatever the reason, if the application on that pod is in the process of handling a request, it's poor form for the pod to immediately shut down, leaving the request unhandled. Doing so will result in an error response to the client and require that the client make the request again.

Instead of burdening the client with an error, you can enable graceful shutdown in your Spring application by simply setting the `server.shutdown` property to `"graceful"`. This can be done in any of the property sources discussed in chapter 6, including in application.yml like this:

```
server:
  shutdown: graceful
```

By enabling graceful shutdown, Spring will hold off on allowing the application to shut down for up to 30 seconds, allowing any in-progress requests to be handled. After all pending requests have been completed or the shutdown time-out expires, the application will be allowed to shut down.

The shutdown time-out is 30 seconds by default, but you can override that by setting the `spring.lifecycle.timeout-per-shutdown-phase` property. For example, to change the time-out to 20 seconds, you would set the property like this:

```
spring:
  lifecycle.timeout-per-shutdown-phase: 20s
```

While the shutdown is pending, the embedded server will stop accepting new requests. This allows for all in-flight requests to be drained before shutdown occurs.

Shutdown isn't the only time when the application may not be able to handle requests. During startup, for example, an application may need a moment to be prepared to handle traffic. One of the ways that a Spring application can indicate to Kubernetes that it isn't ready to handle traffic is with a readiness probe. Next up, we'll take a look at how to enable liveness and readiness probes in a Spring application.

18.3.3 Working with application liveness and readiness

As we saw in chapter 15, the Actuator's health endpoint provides a status on the health of an application. But that health is only in relation to the health of any external dependencies that the application relies on, such as a database or message broker. Even if an application is perfectly healthy with regard to its database connection, that doesn't necessarily mean that it's ready to handle requests or that it is even healthy enough to remain running in its current state.

Kubernetes supports the notion of liveness and readiness probes: indicators of an application's health that help Kubernetes determine whether traffic should be sent to the application, or if the application should be restarted to resolve some issue. Spring Boot supports liveness and readiness probes via the Actuator health endpoint as subsets of the health endpoint known as *health groups*.

Liveness is an indicator of whether an application is healthy enough to continue running without being restarted. If an application indicates that its liveness indicator is down, then the Kubernetes runtime can react to that by terminating the pod that the application is running in and starting a new one in its place.

Readiness, on the other hand, tells Kubernetes whether the application is ready to handle traffic. During startup, for instance, an application may need to perform some initialization before it can start handling requests. During this time, the application's readiness may show that it's down. During this time, the application is still alive, so Kubernetes won't restart it. But Kubernetes will honor the readiness indicator by not sending requests to the application. Once the application has completed initialization, it can set the readiness probe to indicate that it is up, and Kubernetes will be able to route traffic to it.

ENABLING LIVENESS AND READINESS PROBES

To enable liveness and readiness probes in your Spring Boot application, you must set `management.health.probes.enabled` to `true`. In an application.yml file, that will look like this:

```
management:
  health:
    probes:
      enabled: true
```

Once the probes are enabled, a request to the Actuator health endpoint will look something like this (assuming that the application is perfectly healthy):

```
{
  "status": "UP",
  "groups": [
    "liveness",
    "readiness"
  ]
}
```

On its own, the base health endpoint doesn't tell us much about the liveness or readiness of an application. But a request to /actuator/health/liveness or /actuator/health/readiness will provide the liveness and readiness state of the application. In either case, an up status will look like this:

```
{
  "status": "UP"
}
```

On the other hand, if either readiness or liveness is down, then the result will look like this:

```
{
  "status": "DOWN"
}
```

In the case of a down readiness status, Kubernetes will not direct traffic to the application. If the liveness endpoint indicates a down status, then Kubernetes will attempt to remedy the situation by deleting the pod and starting a new instance in its place.

Configuring liveness and readiness probes in the deployment

With the Actuator producing liveness and readiness status on these two endpoints, all we need to do now is tell Kubernetes about them in the deployment manifest. The tail end of the following deployment manifest shows the configuration necessary to let Kubernetes know how to check on liveness and readiness:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: taco-cloud-deploy
  labels:
    app: taco-cloud
spec:
  replicas: 3
```